



TRABAJO DE FIN DE GRADO · DAM

Transitly

Aplicación multiplataforma de transporte público en tiempo real para ciudades medias.

Autor	Koldo Uruburu
Ciclo	Desarrollo de Aplicaciones Multiplataforma
Centro	Victoria FP · 2025/2026
Operador piloto	COMUJESA · Jerez de la Frontera
Versión	v1.12.1
Defensa	2026-06-09
Repositorio	github.com/astralk9999/Transitly

Índice de la memoria

01	Análisis del Contexto y Detección de Necesidades
02	Diseño del Proyecto
03	Planificación de la Ejecución
04	Desarrollo e Implementación
05	Seguimiento, Evaluación y Documentación
06	Manual Técnico
07	Manual de Usuario
08	Presentación Final

01 — Análisis del Contexto y Detección de Necesidades

Proyecto: Transitly (repositorio [nexto-stop-v2](#)). **Autor:** trabajo individual con asistencia documentada de un sistema multiagente de inteligencia artificial. **Ciclo formativo:** Desarrollo de Aplicaciones Multiplataforma (DAM). **Estado verificado:** [master @ b47180d0](#) (8 de junio de 2026); incorpora el ciclo de estabilización y las funcionalidades finales (push FCM real, modo conductor en segundo plano, planificador origen→destino) publicadas en la **release v1.12.1**. Ver §6 y §7 al final del documento para el resumen de cambios.

1. Sector y problema concreto

El sector de referencia es la **movilidad urbana basada en transporte público colectivo en ciudades medias españolas**, con especial atención al autobús urbano e interurbano. Según los datos del Instituto Nacional de Estadística publicados durante 2024, los servicios urbanos de autobús transportaron aproximadamente **1.500 millones de viajeros** y los interurbanos cerca de **600 millones**, lo que confirma una demanda diaria masiva, rutinaria y muy sensible a la calidad de la información ofrecida al usuario.

El sector exhibe tres rasgos estructurales relevantes para cualquier iniciativa tecnológica que pretenda actuar sobre él. El primero es la **demanda masiva y diaria**, en la que el viajero valora más la fiabilidad de la información en tiempo real que el precio del trayecto. El segundo es la **fragmentación operativa**, dado que cada ciudad y cada consorcio cuentan con su propio operador, su propio modelo de datos, su propia aplicación móvil y su propia tarjeta sin contacto, lo que obliga al usuario a familiarizarse de nuevo con cada cambio de territorio. El tercero es la **heterogeneidad tecnológica**: mientras grandes ciudades como Madrid o Barcelona publican feeds GTFS-Realtime y mantienen aplicaciones móviles maduras, las ciudades medias —Jerez de la Frontera, Cuenca, Logroño, Cáceres— se apoyan todavía en paneles web estáticos y aplicaciones mínimas sin información en vivo.

El problema concreto que aborda Transitly es la **asimetría informativa que sufre el usuario del autobús urbano en una ciudad media española**. Concretamente, el viajero no puede saber dónde se encuentra su vehículo, no dispone de estimación de tiempo de llegada, no recibe notificación de incidencias por un canal estructurado, debe gestionar tantas aplicaciones como operadores utilice y solo puede consultar el saldo de su tarjeta NFC del Consorcio de Transportes de Andalucía en máquinas físicas. El caso de estudio piloto es **COMUJESA (Compañía Municipal del Transporte Urbano de Jerez, S.A.)**, operador municipal de Jerez de la Frontera, en la provincia de Cádiz.

2. Tipos de empresas y estructuras

El tejido empresarial del transporte público colectivo en España se articula en torno a cuatro tipologías de actor. Los **operadores municipales** son sociedades de capital público dependientes del ayuntamiento; gestionan flota propia, contratan personal y cuentan con presupuesto subvencionado. COMUJESA en Jerez, EMT en Madrid, Valencia o Málaga, y TUSSEM en Sevilla son ejemplos representativos. Los **consorcios autonómicos o metropolitanos**, como el Consorcio de Transportes de Andalucía o el Consorcio Regional de Transportes de Madrid, agregan a varios operadores municipales e interurbanos, gestionan la tarjeta única zonal e integran las tarifas. Los **operadores privados concesionarios**, adjudicatarios mediante concurso público, explotan rutas interurbanas o municipales con flota propia: Avanza, Damas, AUVASA o TITSA son referentes. Por último, las **aplicaciones agregadoras de terceros** —Moovit, Citymapper o Google Maps Transit— no operan flota, sino que consumen los feeds GTFS publicados por los operadores.

Todos estos actores manejan, con independencia de su forma jurídica, las mismas entidades de negocio: rutas, paradas, horarios, flota, conductores, tarifas e incidencias. Esta convergencia conceptual es la que permite a Transitly proponer un modelo de datos común basado en GTFS y abstraer las diferencias operativas detrás de una única interfaz.

3. Necesidades actuales detectadas

El análisis del estado del arte y la observación directa del caso COMUJESA permiten identificar **siete carencias estructurales** que sufre el viajero de una ciudad media española. La primera es la **ausencia de información de posición en vivo del vehículo**, ya que los autobuses no aparecen en un mapa y el horario estático rara vez refleja la realidad operativa. La segunda es la **inexistencia de estimación de tiempo de llegada**, derivada de la ausencia de GPS a bordo expuesto al usuario. La tercera es la **fragmentación del ecosistema de aplicaciones**, que obliga al usuario a manejar una aplicación distinta por operador con interfaces dispares.

La cuarta carencia es la **accesibilidad deficiente** de las aplicaciones existentes: contrastes insuficientes, ausencia de soporte para personas con dislexia o daltonismo, falta de etiquetado semántico para lectores de pantalla y ausencia de soporte para idiomas con dirección de lectura inversa como el árabe, idioma con presencia creciente entre la población usuaria de transporte público en Andalucía. La quinta es la **opacidad de la tarjeta NFC Mifare Classic** del Consorcio de Transportes de Andalucía, cuyo saldo solo puede consultarse en taquillas, quioscos o máquinas expendedoras físicas. La sexta es la **inexistencia de un canal estructurado para reportar incidencias** —retrasos, averías, desvíos—, que actualmente se vehicula informalmente a través de redes sociales. La séptima y última es la **ausencia de observabilidad** por parte del operador, que no dispone de telemetría agregada del uso de su red.

4. Oportunidades de negocio

El diagnóstico anterior abre **cinco oportunidades de negocio** diferenciables. En primer lugar, un **modelo SaaS B2G** dirigido a operadores municipales y consorcios de tamaño medio, mediante una suscripción anual que incluye marca personalizada, panel de moderación, importador GTFS y telemetría agregada. En segundo lugar, una **plataforma multi-operador** que centralice la experiencia del viajero en una única aplicación con cobertura nacional progresiva, partiendo de Jerez como piloto y escalando al resto de operadores andaluces vía Consorcio. En tercer lugar, una **comunidad UGC** — contenido generado por usuarios— con sistema de reputación, votación y moderación, que aporta la información cualitativa que los feeds oficiales no capturan: averías, incidencias puntuales, sugerencias de rutas inexistentes.

En cuarto lugar, la **accesibilidad como elemento diferenciador** competitivo en un sector donde las aplicaciones dominantes ofrecen una accesibilidad genérica y poco verificada; un cumplimiento documentado de WCAG 2.2 AA constituye una ventaja defendible frente a Google Maps Transit o Moovit. En quinto lugar, la **integración futura con GTFS-Realtime** cuando los operadores expongan estos feeds de manera abierta, lo que permitiría ofrecer información de posición real sin depender del modo conductor interno de la propia aplicación.

5. Guión inicial de trabajo

El guión inicial del trabajo se concreta en la construcción de un **producto mínimo viable de consulta de transporte público, comunidad y accesibilidad**, con COMUJESA como operador piloto en Jerez de la Frontera. La arquitectura se diseña multi-operador desde el primer día, pero el alcance funcional del TFG se limita al operador piloto. El producto incluye consulta de rutas, paradas y horarios, mapa con estimación de posición, lectura del saldo de la tarjeta NFC del Consorcio Andaluz, reporte de incidencias, sistema de reputación comunitaria, modo conductor con grabación GPS, panel administrativo, modo offline con regiones descargables, soporte de accesibilidad multidimensional, internacionalización trilingüe en español, inglés y árabe, y notificaciones push.

Quedan **explícitamente fuera del alcance** del TFG cuatro líneas de trabajo que se documentan como evolución futura: la integración con GTFS-Realtime real (pendiente de que los operadores expongan los feeds), el widget iOS completo (limitado por las restricciones de la plataforma), la expansión activa a otras ciudades distintas de Jerez (requiere acuerdos comerciales con sus respectivos operadores), y el desarrollo de un modelo de aprendizaje automático para predicción precisa de ETA (requiere histórico de posiciones reales que solo se obtiene tras un periodo de operación). Estas exclusiones se recogen formalmente en [docs/EXTERNAL_BLOCKERS.md](#) junto al resto de bloqueadores externos al control del autor.

El documento siguiente, [02_diseno_proyecto.md](#), materializa este análisis en la arquitectura técnica, objetivos funcionales y no funcionales, planificación de recursos y requisitos legales del proyecto.

6. Actualización a 4 de junio de 2026

El análisis de contexto y oportunidades originalmente publicado el 23 de mayo permanece vigente sin cambios estructurales. La validación sobre el terreno tras las dos primeras semanas de iteración refuerza el diagnóstico inicial: la asimetría informativa del viajero en ciudad media sigue siendo el problema central, y la inexistencia de feeds GTFS-Realtime en operadores como COMUJESA continúa siendo la principal barrera a la integración con tiempo real, tal y como se anticipó en

[docs/EXTERNAL_BLOCKERS.md](#).

Las siete carencias detectadas se han abordado todas en mayor o menor medida con la implementación; el grado de cobertura final se documenta en [04_desarrollo_implementacion.md](#) §10 y en [05_evaluacion_documentacion.md](#). La séptima carencia (ausencia de observabilidad) se traslada parcialmente al operador mediante las funciones de panel multi-rol y los paneles de moderación.

No se incorporan nuevas necesidades respecto al análisis inicial: el ámbito funcional se mantiene dentro del catálogo de doce objetivos del documento 02. Los cambios introducidos durante junio responden todos a estabilización (fixes), pulido visual o ampliación de configurabilidad por el usuario (widgets Android, wizard de rutas con tap en mapa), sin alterar el alcance del TFG.

7. Cierre a 8 de junio de 2026 (preparación de la defensa)

En la recta final previa a la defensa se cerraron las tres carencias que aún se cubrían de forma parcial, todas dentro del alcance original sin ampliarlo:

1. **Notificaciones push reales (carencia 6, canal de incidencias).** Se completó la integración de **Firestore Cloud Messaging**: la aplicación recibe avisos con la app cerrada y registra el token del dispositivo en [device_tokens](#) al iniciar sesión. El backend de envío (Edge Function [send_notification](#) sobre FCM HTTP v1 más los disparadores SQL) ya existía; solo restaba el cableado del cliente y la *service account* del operador, documentada como dependencia externa en [docs/FCM_SETUP.md](#).
2. **Posición en vivo del vehículo (carencias 1 y 2).** El **modo conductor** pasó a compartir la ubicación mediante un *foreground service* de Android, de modo que el seguimiento continúa con la pantalla bloqueada o la app en segundo plano, reforzando la fiabilidad de la posición en el mapa para el resto de viajeros.
3. **Planificación de trayecto origen→destino.** Se reactivó el planificador de rutas con transbordos, cerrando el bucle de "consulta" de la carencia informativa.

Estos cambios se distribuyen en la **release pública v1.12.1** (APK universal, Android 7.0+) publicada en GitHub Releases. El diagnóstico de contexto y oportunidades permanece, por tanto, plenamente vigente y ahora respaldado por una implementación que cubre las siete carencias detectadas.

02 — Diseño del Proyecto

Proyecto: Transitly (repositorio `nexto-stop-v2`). **Estado verificado original:** `master @ b908f3c` (23 de mayo de 2026). **Estado actualizado:** `master @ b47180d0` (8 de junio de 2026); **release pública v1.12.1** con **APK universal** distribuida en GitHub Releases (<https://github.com/astalk9999/Transitly/releases/tag/v1.12.1>). Métricas verificadas a fecha de defensa: **679 tests** (0 fallos), **51 migraciones SQL** consecutivas, **27 features**, **8 Edge Functions** desplegadas, **446 ficheros .dart** (~94.000 líneas, sin contar código generado), **30 modelos freezed**, **38 widgets compartidos**, **642 claves ARB** en español (ES/EN/AR). Scorecard maestro: **TFG 8,9/10 · Producción 6,0/10** (`docs/00_MAESTRO.md`). Ver §9 y §10 al final del documento para el resumen consolidado de los cambios de junio.

1. Información técnica recopilada

Transitly se sustenta sobre un conjunto de **estándares abiertos** y **especificaciones normativas** que se han estudiado y consolidado antes de iniciar la fase de implementación. El estándar de datos estáticos es **GTFS** (General Transit Feed Specification), formato canónico publicado por Google y la fundación MobilityData para describir redes de transporte público; el feed estático contiene agencias, rutas, paradas, viajes y calendarios. Su complemento dinámico, **GTFS-Realtime**, expone posiciones de vehículo, predicciones de paso y alertas de servicio mediante mensajes Protobuf consumibles vía HTTP; no se integra aún por no estar disponible en COMUJESA, pero el modelo de datos lo contempla.

La cartografía se apoya en **OpenStreetMap** mediante capas vectoriales servidas por proveedores compatibles (MapTiler como primario, CartoDB como respaldo). La interoperación con la tarjeta de transporte se realiza sobre **NFC Mifare Classic 1K**, tecnología empleada por el Consorcio de Transportes de Andalucía y leída en modo solo lectura, lo que evita la necesidad de certificación bancaria. Las notificaciones se vehiculan por **Firestore Cloud Messaging** sobre la API HTTP v1 con autenticación OAuth JWT. La seguridad a nivel de fila se delega en el motor de **Row Level Security de PostgreSQL** con política DENY-by-default.

En cuanto al marco normativo, el proyecto se diseña en conformidad con la **WCAG 2.2 nivel AA**, con el **Reglamento (UE) 2016/679 (RGPD)** y su transposición nacional **LOPDGDD (Ley Orgánica 3/2018)**, con el **Real Decreto 1112/2018** sobre accesibilidad del sector público, y con el **Reglamento (UE) 2022/2065 (Digital Services Act)** en lo relativo a obligaciones de transparencia para plataformas con contenido generado por usuarios.

2. Objetivos funcionales

Se han fijado **doce objetivos funcionales** con sus respectivos criterios de aceptación verificables.

1. **Consulta de rutas, paradas y horarios** del operador piloto COMUJESA; aceptación: el listado completo se renderiza en menos de un segundo desde caché Hive.
2. **Visualización del autobús en mapa con estimación de posición**; aceptación: la posición se interpola a partir del horario y de la posición real del conductor cuando esta existe en el canal Realtime.
3. **Lectura del saldo de la tarjeta NFC del Consorcio Andaluz** mediante Mifare Classic; aceptación: el saldo se muestra en menos de tres segundos tras el contacto físico.
4. **Reporte de incidencias** geolocalizadas con votación de la comunidad; aceptación: la incidencia se persiste en `incidents` con su geometría PostGIS y se replica vía Realtime.
5. **Feedback sobre rutas existentes** con sistema de rangos y reputación; aceptación: el voto se asocia al perfil autenticado y respeta una política de un voto por usuario y ruta.
6. **Sugerencia de rutas faltantes** por parte de los usuarios; aceptación: la propuesta entra en el panel de moderación y exige aprobación.
7. **Modo conductor con grabación GPS** activable por código de invitación; aceptación: el conductor emite posición al canal Realtime con cadencia configurable.
8. **Panel de administración multi-rol** (pasajero, conductor, administrador de operador, moderador, administrador global); aceptación: el guard del router consume `profiles.role` desde Supabase.
9. **Modo offline** con regiones descargables y cola FIFO de acciones pendientes; aceptación: las acciones se drenan al recuperar conectividad con backoff exponencial.
10. **Accesibilidad ajustable** —contraste configurable, soporte de daltonismo y dislexia, reduce-motion, escala de texto, objetivos táctiles de 48 dp—; aceptación: los ajustes persisten y aplican sin reinicio.
11. **Internacionalización ES/EN/AR** con soporte completo de dirección de lectura inversa; aceptación: las 642 claves ARB existen en español, con cobertura en inglés y árabe.
12. **Notificaciones push** con horas silenciosas configurables; aceptación: la entrega se gobierna por consentimiento explícito y `quiet_hours`. Verificado en v1.12.1 con Firebase Cloud Messaging real (recepción con app cerrada y registro de token en `device_tokens`).

3. Objetivos no funcionales

Los objetivos no funcionales se agrupan en siete familias. En **rendimiento**, el arranque en frío del percentil 95 se sitúa por debajo de 2 segundos y la transición entre pestañas del percentil 95 por debajo de 300 milisegundos. En **escalabilidad**, la arquitectura soporta 10.000 usuarios activos diarios sin refactor y 100.000 con la incorporación de *clustering* en el mapa.

En **accesibilidad** se persigue la conformidad WCAG 2.2 AA verificable estáticamente sobre el código; el acta de pruebas con productos de apoyo (TalkBack y VoiceOver) se programa para la semana 10. En **seguridad** se aplica RLS DENY-by-default sobre todas las tablas, cifrado en reposo de las cajas Hive sensibles mediante `HiveAesCipher`, y almacenamiento de claves criptográficas en el llavero del sistema operativo a través de `flutter_secure_storage` (Keychain en iOS, Keystore en Android).

En **privacidad** la política es opt-out por defecto, con consentimiento granular y persistente; se implementan el artículo 8 (verificación de edad), el artículo 13 (información al interesado), el artículo

17 (derecho de supresión mediante un worker de borrado), el artículo 20 (portabilidad por exportación) y el artículo 21 (revocación de consentimiento en caliente sin reinicio) del RGPD. En **observabilidad** se instrumentan **6 spans** de Sentry y **17 eventos** de PostHog, todos gobernados por consent-gating. Finalmente, en **mantenibilidad e internacionalización**, la arquitectura limpia se acompaña de **679 tests automatizados**, **6 jobs de CI en verde**, RTL para árabe y `flutter analyze` con cero issues bloqueantes.

4. Fases y cronograma

El proyecto se desarrolla en un total de **11 semanas**, del 1 de abril al 16 de junio de 2026, organizadas en seis bloques. Las **semanas 1 y 2** (1–14 de abril) se dedican al análisis del contexto y a la detección de necesidades, materializadas en el documento 01. Las **semanas 3 y 4** (15–28 de abril) abarcan el diseño del proyecto —arquitectura, modelo de datos y objetivos— recogido en el presente documento. La **semana 5** (29 de abril–5 de mayo) corresponde a la planificación de la ejecución, con la primera entrega parcial el 5 de mayo. Las **semanas 6 a 9** (6 de mayo–2 de junio) constituyen el desarrollo e implementación, con la segunda entrega parcial el 2 de junio. La **semana 10** (3–9 de junio) se reserva al seguimiento, evaluación y elaboración de los manuales de usuario y desarrollador, incluida el acta de accesibilidad con productos de apoyo. La **semana 11** (10–16 de junio) se destina a la preparación y defensa pública del trabajo.

Las dos entregas parciales —fin de planificación el 5 de mayo y fin de desarrollo el 2 de junio— actúan como hitos de control. El detalle completo del Gantt y de las dependencias internas se recoge en `docs/tfg/03_planificacion.md`.

5. Estudio de viabilidad técnica

La elección de **Flutter 3.x con Dart 3** se justifica por cuatro razones: lenguaje único para móvil, web y escritorio que reduce a la mitad el esfuerzo de mantenimiento; *hot reload* que acelera el ciclo de iteración; un ecosistema maduro con respaldo de Google y de un consorcio amplio de empresas; y un rendimiento de renderizado nativo basado en Skia/Impeller. La alternativa razonable era React Native; se descartó por la fragmentación del ecosistema de navegadores de estado y por la menor cohesión de su capa nativa.

La elección de **Supabase** combina en un único servicio gestionado un PostgreSQL serio con soporte RLS, un servicio de autenticación, Edge Functions sobre Deno con tiempos de arranque del orden de decenas de milisegundos, suscripciones Realtime sobre WebSocket y almacenamiento de objetos. La alternativa Firebase puro se descartó por su mayor *vendor lock-in* y por la imposibilidad de ejecutar consultas SQL sobre el modelo de datos. La elección de **Riverpod 2.6** se motiva por su naturaleza compilable, su testeabilidad sin necesidad de `BuildContext` y su capacidad de generar proveedores con `autoDispose` de manera segura; la alternativa Bloc se descartó por el mayor *boilerplate* requerido. **Hive 2.2** se selecciona como almacén clave-valor por su rapidez y por la disponibilidad de cifrado AES integrado en las cajas sensibles; Realm se descartó por la pobreza relativa de su

ecosistema en Flutter. La arquitectura **feature-first** facilita el crecimiento horizontal del equipo a futuro y aísla los dominios funcionales.

Las dependencias auxiliares completan el cuadro: **Firebase Messaging 16** para notificaciones push, **Sentry 8** para reporte de fallos, **PostHog 5** para analítica de producto, **flutter_secure_storage** para almacenamiento de claves criptográficas, **very_good_analysis** y **leak_tracker_flutter_testing** para enforcement de calidad estática y detección de fugas de memoria en tests.

6. Planificación de actividades, recursos, personal y financiación

Las **actividades** se detallan en [docs/tfg/03_planificacion.md](#) y se agrupan en análisis, diseño, planificación, implementación por bloques (cimientos, identidad, datos, experiencia, comunidad, infraestructura, plataformas y cierre) y seguimiento.

Los **recursos humanos** se reducen a un único desarrollador en régimen individual, con el respaldo del tutor académico del módulo y un grupo de cinco usuarios de prueba para la semana 10. La asistencia del sistema multiagente de inteligencia artificial queda declarada con transparencia en [multiagent/ARCHITECTURE.md](#) por integridad académica. Los **recursos técnicos** comprenden un equipo portátil de desarrollo, un emulador Android Pixel 6 con API 34, una cuenta gratuita de Supabase, una cuenta Firebase en plan Spark, un repositorio de GitHub gratuito y cuentas gratuitas de Sentry y PostHog. La **financiación directa** asciende a 0 euros durante el ciclo del TFG, dado que todas las herramientas se utilizan en sus planes gratuitos y la publicación pública en tiendas (25 euros únicos en Google Play y 99 euros anuales en Apple Developer) queda fuera del alcance académico.

7. Indicadores de calidad

Se han definido **nueve indicadores medibles** con sus respectivos umbrales y estado verificado. La cobertura de tests se fija como objetivo en el 60 % y se encuentra actualmente en el 24 %; la palanca correctiva, identificada y activa, consiste en escribir tests para la capa [remote/](#). El análisis estático con [flutter analyze](#) debe mantenerse en cero issues bloqueantes. El tamaño objetivo por ABI es inferior a 50 MB; el APK release v1.12.1 se distribuye como **APK universal de ~91 MB** (incluye arm64-v8a, armeabi-v7a y x86_64 en un solo binario para máxima compatibilidad), deuda de tamaño explicitada en el scorecard y mitigable con separación por ABI o App Bundle. La accesibilidad WCAG 2.2 AA se verifica estáticamente y se complementará con acta manual. Los *advisors* de seguridad de Supabase deben permanecer en cero. La latencia p95 del flujo de autenticación se limita a 800 milisegundos y la del renderizado inicial del mapa a 1.500 milisegundos. Los 6 jobs de GitHub Actions deben permanecer en verde y el escáner Semgrep actúa de modo bloqueante en *pull request*.

8. Requisitos legales y conformidad

El proyecto se ajusta a un marco normativo amplio. En materia de **protección de datos** se aplican los artículos del RGPD ya mencionados —5 (minimización), 6 (base legal), 7 (consentimiento), 8

(menores), 13 (información), 17 (supresión), 20 (portabilidad) y 21 (oposición)— en conjunción con la LOPDGDD. En materia de **plataformas digitales** se observa el Reglamento de Servicios Digitales en lo relativo a transparencia, reporte de contenido ilícito y dotación de mecanismos de notificación y reclamación accesibles al usuario.

Respecto a las **licencias de software libre**, las dependencias utilizadas operan bajo licencias permisivas compatibles con uso comercial: **MIT** (Flutter, Riverpod, Hive, freezed, go_router, sentry_flutter), **BSD** (Dart, flutter_local_notifications) y **Apache 2.0** (plugins de Firebase, flutter_map, posthog_flutter). Los datos GTFS de COMUJESA se utilizan bajo licencia de uso público consultada en fuentes oficiales del Ayuntamiento de Jerez. Las tipografías DM Sans e IBM Plex Mono se distribuyen bajo SIL Open Font License y se bundlean como assets locales para evitar peticiones en tiempo de ejecución y preservar la privacidad del usuario. Los iconos Lucide se utilizan bajo licencia ISC.

Por último, el proyecto incluye **Términos del Servicio** y **Política de Privacidad** propias, alojadas en `assets/legal/` en versiones trilingües (español, inglés y árabe), accesibles desde el panel de privacidad de la aplicación. El documento siguiente, `03_planificacion.md`, desarrolla con mayor detalle el Gantt y la asignación temporal de actividades.

9. Actualización a 4 de junio de 2026

El diseño del proyecto presentado en este documento se mantiene íntegro: la pila tecnológica, los doce objetivos funcionales, los objetivos no funcionales agrupados en siete familias, el cronograma de 11 semanas y los nueve indicadores de calidad continúan siendo el marco de referencia. Los cambios introducidos entre el 23 de mayo y el 4 de junio responden todos a refuerzo, estabilización o extensión incremental dentro del alcance ya planificado, sin alterar ninguna decisión arquitectónica de fondo.

Cambios significativos a nivel de arquitectura:

- 1. Recovery boot y persistencia diferida de preferencias:** se introdujo `BootCanary` (`lib/core/utils/boot_canary.dart`) y `RecoveryScreen` (`lib/features/recovery/`) como red de seguridad ante crashes nativos del engine de Flutter al combinar opciones de accesibilidad (dislexia + alto contraste + tamaño de texto). El mecanismo persiste sólo tras el primer frame estable y revierte la última preferencia tóxica si se detectan dos crashes consecutivos en arranque. Esto refuerza el objetivo no funcional de mantenibilidad sin alterar el modelo de datos.
- 2. Migración SQL adicional `fix_route_shares_rls_recursion`:** se detectó un ciclo entre las políticas RLS de `route_shares` y `routes` que provocaba error PostgreSQL **42P17** (infinite recursion in policy) al consultar las contribuciones del usuario. Se resolvió mediante una función `SECURITY DEFINER` `public.is_route_owner(uuid)` que bypassa RLS para el lookup, manteniendo la semántica original de visibilidad. Documentado en `docs/SUPABASE_SETUP.md`.
- 3. Bypass temporal de verificación de email:** ante la imposibilidad de configurar SMTP propio dentro del cronograma del TFG, se desactivó la verificación obligatoria de correo en el listener de auth (`auth_repository_supabase.dart`) y se añadió un auto-login defensivo en `signupWithEmail`. La infraestructura `EmailVerifyPendingScreen` queda dormida y reactivable cuando se configure SMTP. Documentado en `docs/SUPABASE_SETUP.md`.

4. **Filtros del mapa con árbol jerárquico:** el filtro plano de operadores se substituyó por un árbol expandible de tres niveles (zona → compañía → líneas) con checkbox tri-state. Refuerza el objetivo de escalabilidad multi-operador del cronograma futuro.
5. **Widgets Android configurables:** pantallas de configuración en perfil para los tres widgets (Próximo bus, Mi línea, Saldo NFC) con preview y botón "Probar". Implementación en `lib/features/widgets_config/`.
6. **Wizard de creación de rutas con tap en mapa:** el formulario manual de coordenadas se complementa con `MapStopPickerScreen` que permite añadir paradas tocando el mapa y buscar lugares vía Nominatim (OpenStreetMap).
7. **Release pública v1.11.0:** el APK release se publica como asset en GitHub Releases (`https://github.com/astral9999/Transitly/releases/tag/v1.11.0`). La presentación web ahora enlaza a `releases/latest` en lugar de versionar APKs en el repositorio, lo que ha reducido el `working tree` en ~792 MB y previene el bloqueo de push por archivos >100 MB.

Los demás aspectos del diseño —modelo de datos, objetivos de rendimiento, cronograma, indicadores de calidad y marco legal— continúan vigentes sin modificaciones.

10. Actualización a 8 de junio de 2026 (cierre y release v1.12.1)

En los días previos a la defensa se consolidaron las funcionalidades que cerraban los objetivos funcionales 2, 7 y 12, junto con el pulido de la web de presentación. Ninguna decisión arquitectónica de fondo se vio alterada; los cambios son extensión incremental y estabilización.

1. **Notificaciones push reales con FCM (objetivo funcional 12).** Se completó el cliente Firebase Cloud Messaging: `google-services.json` del proyecto `transitly-ee8cf`, plugin Gradle `com.google.gms.google-services`, claves reales en `firebase_options.dart`, registro de los manejadores de mensajes en segundo y primer plano, y persistencia del token del dispositivo en `device_tokens` al iniciar sesión (`auth_repository_supabase.dart`). El pipeline servidor —Edge Function `send_notification` (OAuth JWT → FCM HTTP v1, con limpieza de tokens inválidos) y disparadores SQL (`010_push_triggers.sql`)— ya estaba construido. La única dependencia externa pendiente, la *service account* del operador para el envío programático, queda documentada en `docs/FCM_SETUP.md`.
2. **Modo conductor en segundo plano (objetivo funcional 7).** El seguimiento GPS del conductor pasó de un temporizador en primer plano a `Geolocator.getPositionStream` con un **foreground service** de Android (`ForegroundNotificationConfig`, wake-lock, notificación persistente). La posición sigue emitiéndose al canal Realtime con la app en segundo plano o la pantalla bloqueada. Permisos `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION` y `WAKE_LOCK` añadidos al manifiesto.
3. **Planificador origen→destino (objetivo funcional 1/2).** Se reactivó la pestaña de búsqueda y el flujo origen→destino con transbordos (`RoutePlannerService`), navegando a `/route-plan`.
4. **Web de presentación.** Corrección del modo claro (velo de contraste y blobs adaptables al tema) y publicación del sitio en GitHub Pages, con descarga del APK resuelta automáticamente desde la release más reciente.

5. **Release pública v1.12.1. APK universal** (arm64-v8a, armeabi-v7a, x86_64), Android 7.0+, publicado como asset en GitHub Releases. Sustituye al anterior APK por ABI única (solo arm64) y al v1.11.0.

Los demás aspectos del diseño continúan vigentes sin modificaciones.

03 — Planificación de la Ejecución

Proyecto: Transitly **Repositorio:** nexto-stop-v2 (anclaje original `master @ b908f3c` · 2026-05-23; estado actualizado `master @ b47180d0` · 2026-06-08, release v1.12.1) **Ciclo formativo:** Desarrollo de Aplicaciones Multiplataforma (DAM) **Sector:** movilidad urbana — operador COMUJESA (Jerez de la Frontera) **Cronograma:** 11 semanas naturales, del 01/04/2026 al 16/06/2026 **Autor:** estudiante individual, con asistencia de IA documentada con transparencia académica

1. Metodología

1.1. Marco general: Scrum solo adaptado

El proyecto se desarrolla por un único autor, lo que impide aplicar Scrum en su forma canónica (no existen Product Owner, Scrum Master y equipo diferenciados). Se adopta por tanto una variante reconocida en la literatura como **Scrum solo**: la persona desarrolladora asume los tres roles, y se preservan únicamente los artefactos que aportan valor real a un proyecto individual.

Artefacto Scrum oficial	Aplicación en Transitly
Sprint	Iteración semanal cerrada (lunes a domingo).
Sprint planning	Sesión de planificación lunes (45 minutos) que selecciona ítems del backlog.
Daily stand-up	Auto-stand-up diario asíncrono escrito en <code>CHANGELOG.md</code> y commits convencionales.
Sprint review	Demostración al tutor cada dos semanas; verificación con CI verde como criterio de aceptación.
Sprint retrospective	Retrospectiva escrita cada dos sprints en <code>docs/retros/</code> (qué funcionó, qué no, acciones).
Backlog	<code>docs/00_MAESTRO.md</code> y plan mega refinamiento priorizado (P0-P3).

1.2. Fases iniciales en cascada

Las fases de **Análisis** (semanas 1-2) y **Diseño** (semanas 3-4) se ejecutan en **modelo cascada**, no iterativo, por dos motivos acreditables:

1. La normativa del TFG exige documentación previa estable antes de programar (memoria del proyecto, especificación de requisitos).
2. Las decisiones arquitectónicas estructurales (stack, modelo de datos, patrón de capas) deben fijarse antes de comenzar el desarrollo para evitar retrabajo masivo.

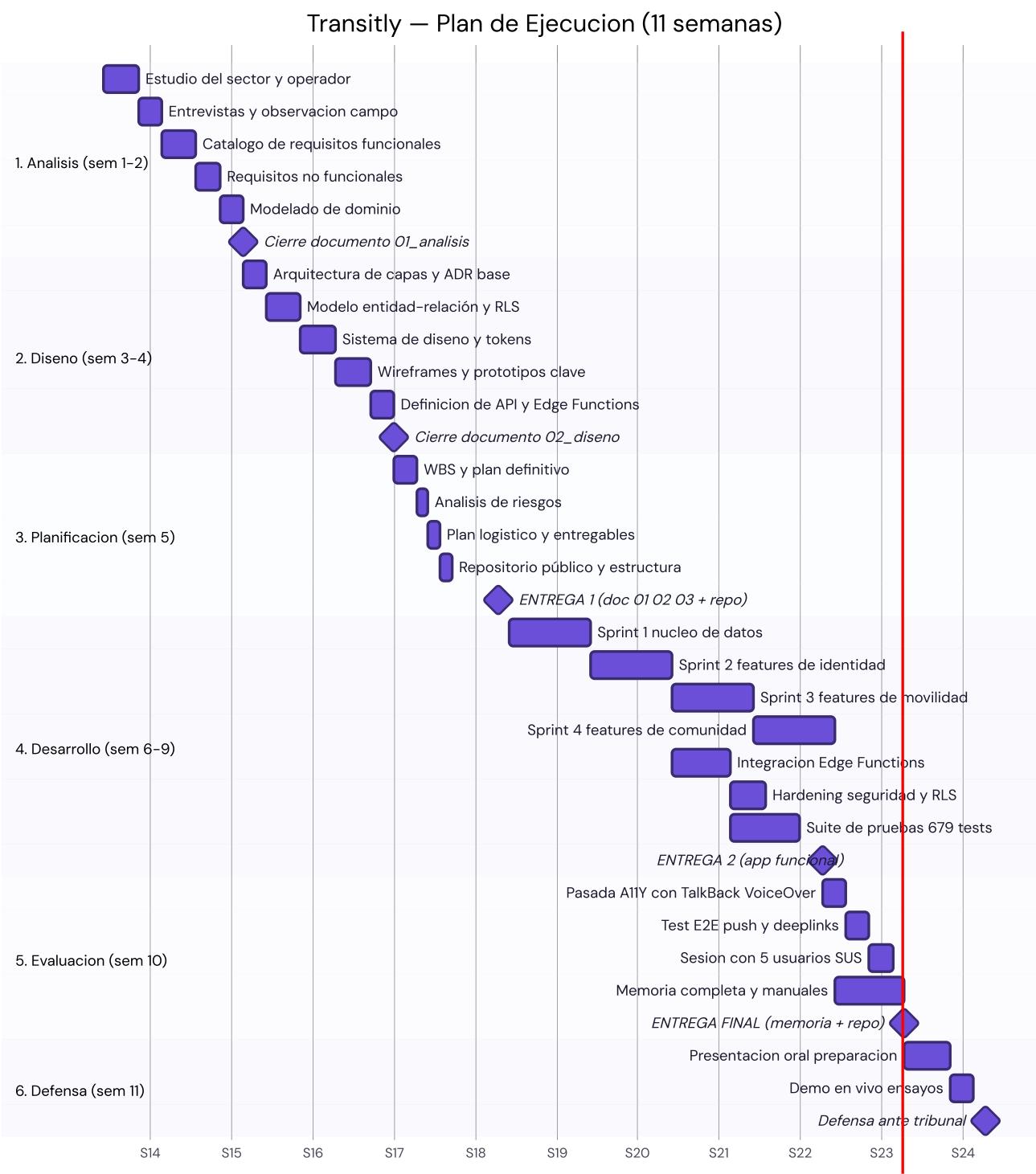
Las semanas 6 a 9 (Desarrollo) sí se ejecutan en sprints semanales iterativos. La semana 5 (Planificación) cierra los entregables previos al desarrollo. Las semanas 10 (Evaluación) y 11

(Defensa) son fases secuenciales.

1.3. Reglas operativas

- **Conventional Commits** obligatorios (`feat:`, `fix:`, `docs:`, `test:`, `chore:`, `refactor:`, `perf:`, `ci:`, `build:`, `revert:`).
 - **CI verde como criterio de mergeo:** cuatro trabajos en GitHub Actions (analyze, test, build-web, build-android) más Gitleaks y Semgrep deben pasar antes de avanzar de iteración.
 - **Auto-revisión por *pull request*** sobre `master` : aunque el autor sea único, todo cambio sustantivo se introduce vía PR para que CI ejecute las verificaciones automáticas y para dejar un rastro auditable.
-

2. Diagrama de Gantt



El diagrama anterior se renderiza con Mermaid (sintaxis `gantt`). Cuatro hitos principales marcan los entregables: ENTREGA 1 (5 de mayo de 2026), ENTREGA 2 (2 de junio de 2026), ENTREGA FINAL (9 de junio de 2026) y la defensa ante el tribunal en la semana 11.

3. Desglose de tareas (WBS)

3.1. Fase 1 — Análisis (semanas 1-2)

Tarea	Duración	Producto
Estudio del sector y operador (COMUJESA, GTFS, normativa)	3 días	Marco teórico
Entrevistas y observación de campo	2 días	Notas de campo
Catálogo de requisitos funcionales (RF01-RFnn)	3 días	Tabla RF
Requisitos no funcionales (rendimiento, A11Y, seguridad, GDPR)	2 días	Tabla RNF
Modelado de dominio (UML, glosario)	2 días	Modelo

3.2. Fase 2 — Diseño (semanas 3-4)

Tarea	Duración	Producto
Arquitectura de capas y ADR base (5 ADRs)	2 días	docs/adr/
Modelo entidad-relación + políticas RLS	3 días	DER + RLS plan
Sistema de diseño y <i>design tokens</i>	3 días	core/theme/
Wireframes y prototipos de pantallas críticas	3 días	Figma / mocks
Definición de API y Edge Functions	2 días	Especificación

3.3. Fase 3 — Planificación (semana 5)

Tarea	Duración	Producto
Gantt definitivo y WBS	2 días	Este documento
Análisis de riesgos y plan de contingencia	1 día	Tabla §6
Plan logístico y entregables parciales	1 día	§7
Repositorio público y estructura inicial	1 día	Repo

3.4. Fase 4 — Desarrollo (semanas 6-9)

Sprint	Duración	Producto
Sprint 1: núcleo de datos (Hive, Supabase wiring, repos canónicos)	7 días	6 dominios cableados
Sprint 2: identidad (auth, roles, perfiles, NFC)	7 días	Auth + NFC
Sprint 3: movilidad (mapa, paradas, rutas, horarios, GPS)	7 días	Núcleo funcional
Sprint 4: comunidad y admin (incidencias, votos, panel)	7 días	App completa
Integración Edge Functions (4 funciones Deno)	5 días	Backend en servicio

Sprint	Duración	Producto
<i>Hardening</i> seguridad: RLS DENY-by-default, scrubbing	3 días	Capa segura
Suite de pruebas (objetivo verificado: 619 tests)	6 días	Tests verdes

3.5. Fase 5 — Evaluación (semana 10)

Tarea	Duración	Producto
Pasada manual A11Y con TalkBack y VoiceOver	2 días	docs/A11Y_MANUAL_TEST_2026_06.md
Test E2E push (foreground / background / killed) y <i>deeplinks</i>	2 días	Informe E2E
Sesión con 5 usuarios reales + cuestionario SUS	2 días	Resultados SUS
Memoria completa (8 documentos TFG) y manuales	6 días	Memoria

3.6. Fase 6 — Defensa (semana 11)

Tarea	Duración	Producto
Preparación de presentación oral	4 días	Diapositivas
Ensayo de demo en vivo	2 días	Demo ensayada
Defensa ante tribunal	1 día	Acto formal

4. Recursos humanos

4.1. Equipo

Rol	Responsabilidades	Dedicación
Desarrolladora / desarrollador único	Análisis, diseño, codificación, pruebas, documentación, despliegue, defensa	Tiempo prácticamente completo durante 11 semanas
Tutor académico	Revisión bi-semanal, validación de hitos, orientación metodológica	1 hora cada dos semanas + correo
Asistencia IA documentada	Apoyo en redacción, revisión de código, generación de pruebas; declarada con transparencia y trazada por commit	Por sesión

4.2. Usuarios de prueba (semana 10)

Se reclutan **cinco perfiles diversos** para la sesión de evaluación de usabilidad con cuestionario SUS:

Perfil	Criterio de selección	Foco de la prueba
Usuaría habitual de COMUJESA, 65 años	Representa al colectivo senior, baja alfabetización digital	Tamaño de tipografías, claridad de iconos, flujo de pago
Estudiante, 20 años, usuaria de TalkBack	Discapacidad visual; lector de pantalla activado	Cobertura semántica, orden de foco, etiquetas
Conductor profesional, 45 años	Usuario del rol <i>driver</i>	Activación con código de invitación, modo en vivo
Turista anglohablante (sin español)	Localización al inglés y a árabe	Traducción de cadenas, <i>fallbacks</i>
Persona en silla de ruedas	Ergonomía móvil con una sola mano	Distancia de pulsadores, <i>touch targets</i>

5. Recursos técnicos y financieros

5.1. Recursos técnicos

Categoría	Detalle
Equipo de desarrollo	Portátil personal (16 GB RAM, SSD), Windows 10 Pro
IDE y <i>toolchain</i>	Visual Studio Code, Android Studio (para emulador), Flutter 3.x, Dart 3, <code>dart_pub</code> , <code>build_runner</code>
Emulador Android	Pixel 6, API 34, perfil de pantalla 6,4"
Dispositivo físico Android	Necesario para pruebas de NFC y <i>push en background / killed</i>
Servicio backend	Cuenta Supabase plan <i>Free</i> (PostgreSQL, Auth, Edge Functions, Storage)
Mensajería <i>push</i>	Firebase Spark (gratis, sin tarjeta)
Repositorio y CI	GitHub plan <i>Free</i> con GitHub Actions (6 <i>jobs</i> : analyze, test, build-web, build-apk, semgrep, gitleaks)
Observabilidad	Sentry plan <i>Free</i> (errores) y PostHog plan <i>Free</i> (eventos)
Sitio web del proyecto	GitHub Pages (presentación + entregables del TFG + descarga del APK)

5.2. Recursos financieros

Concepto	Coste directo	Coste oculto
Servicios <i>cloud</i> citados	0 €	Limitaciones de cuota; bloqueadores documentados
Horas de la persona desarrolladora	0 € (no facturadas)	Aproximadamente 300 horas

Concepto	Coste directo	Coste oculto
Electricidad y conectividad doméstica	0 € (asumido)	Coste marginal
Licencia Apple Developer	No aplica (se entrega para Android)	—
Licencia Google Play Console	No aplica para el TFG (despliegue se aborda como trabajo futuro)	—

Coste directo total: 0 €. El coste oculto principal son las horas del autor.

6. Análisis de riesgos y contingencias

La probabilidad se etiqueta como Baja (B), Media (M) o Alta (A). El impacto sigue la misma escala. Las contingencias se activan cuando el riesgo se materializa; las mitigaciones se aplican de forma preventiva.

#	Riesgo	Prob.	Impacto	Mitigación	Contingencia
R01	Pérdida del <i>keystore</i> Android (impide firmar futuras versiones)	B	A	Tres copias: USB cifrado, almacenamiento cloud privado, copia impresa en caja fuerte; documentado en docs/runbooks/	Si falla todo, generar nuevo <i>keystore</i> y notificar la rotación de huella SHA-256; aceptar que la app figura como nueva en <i>stores</i>
R02	Dependencia de Firebase Cloud Messaging (FCM) por Google	M	A	Abstracción en send_notification Edge Function; APNs preparado como alternativa iOS	Reemplazar capa FCM por OneSignal o un servicio equivalente; aislar cambio en una sola función
R03	Cuenta Supabase pausada por inactividad (<i>free tier</i>)	A	A	Acceso semanal forzado; <i>cron</i> semanal con GET /health ; documentado en runbook	Migración manual a otra cuenta o restauración mediante migraciones SQL versionadas (001-013 y 016)
R04	<i>Drift</i> documental: docs y código divergen	A	M	Revisión cruzada en cada PR; docs/00_MAESTRO.md como única fuente de verdad; <i>checklist</i> de cierre de sprint	Sprint dedicado de re-sincronización antes de la entrega final
R05	<i>Scope creep</i> (alcance excesivo)	A	A	<i>Backlog</i> priorizado (P0-P3) y plan mega refinamiento con criterios de cierre; 19 bloqueadores externos ya aceptados como diferidos	Cortar funcionalidad por debajo de P1 y marcarla como trabajo futuro en la memoria

#	Riesgo	Prob.	Impacto	Mitigación	Contingencia
R06	Falta de usuarios reales para la sesión de prueba	M	M	Cinco perfiles convocados con dos semanas de antelación; mensajes de recordatorio	Si fallan, sustituir por compañeras o compañeros del ciclo con perfil similar y registrar la limitación honestamente
R07	Plagio involuntario derivado de asistencia con IA	M	A	Cada contribución se revisa, atribuye y firma como autoría propia; <i>prompts</i> y respuestas relevantes archivados	Pasar la memoria y el código por una herramienta antiplagio antes de la entrega final
R08	Problemas reales de accesibilidad detectados con TalkBack	M	M	Tests automatizados con <code>Semantics</code> ; matrices de daltonismo incorporadas	Ciclo de corrección priorizado P0 antes de la entrega final
R09	Bug bloqueante descubierto en la última semana	M	A	CI exhaustiva durante todo el desarrollo; <i>smoke tests</i> nocturnos; <i>feature flags</i>	Aislar la funcionalidad afectada con <i>feature flag</i> y desactivar; documentar como limitación conocida
R10	Dependencia única (<i>bus-factor</i> 1)	A	A	Documentación exhaustiva (5 ADRs, 6 <i>runbooks</i> , manuales técnico y de usuario) para que la memoria sobreviva al autor	El proyecto está diseñado para ser legible sin el autor: 73+ documentos en <code>docs/</code>
R11	Retraso por refactor mayor (por ejemplo, migración a Riverpod 3)	B	M	Versiones fijadas en <code>pubspec.yaml</code> (Riverpod 2.6, GoRouter 17.2, Sentry 8.14, Freezed 3); no se hacen <i>upgrades</i> especulativos en plena ejecución	Posponer la migración a trabajo futuro y dejarla recogida en plan mega refinamiento
R12	Fallos en CI por cambios en <i>runners</i> de GitHub	B	M	<code>ubuntu-latest</code> fijado como objetivo; versiones de acciones ancladas	Anclar versión del <i>runner</i> o usar imagen autoalojada si el fallo persiste
R13	Dependencias OSS abandonadas o vulnerables	M	M	Auditoría con <code>flutter pub outdated</code> y <code>gitleaks</code> ; lista de dependencias en <code>pubspec.yaml</code> revisada al inicio de cada sprint	Fork temporal con parche o sustitución por alternativa; documentar en ADR

6.1. Plan de contingencia para riesgos críticos

Tres riesgos requieren un plan ampliado porque su materialización pondría en peligro la entrega:

Pérdida del *keystore* (R01). Se mantiene una copia cifrada en un USB físico guardado en un domicilio distinto al de trabajo, una segunda copia en un *bucket* privado con cifrado en reposo, y una copia impresa de la *passphrase* (no del archivo) en una caja fuerte familiar. En caso de pérdida total, el

procedimiento de emergencia documentado en `docs/runbooks/` describe la generación de un nuevo *keystore* y la comunicación de la nueva huella SHA-256 al tutor.

Cuenta Supabase pausada (R03). El *free tier* suspende proyectos sin actividad. Se programa un acceso forzado semanal y un *cron* externo (Vercel) que llama al endpoint de salud. Si el proyecto se pausa, las 14 migraciones SQL versionadas (001 a 013 y 016) permiten recrearlo en otra cuenta en menos de una hora.

Bug bloqueante en la última semana (R09). Toda funcionalidad candidata a romperse está aislada tras un *feature flag* (consulta de *remote config* o variable `--dart-define`). En caso de bloqueo, se desactiva el *flag*, se retira la funcionalidad de la *build* de defensa y se documenta como limitación conocida en la memoria.

7. Necesidades logísticas

Necesidad	Cobertura
Copia de seguridad diaria en la nube	Repositorio Git con <i>push</i> diario a GitHub más copia automática del directorio <code>docs/</code> a Google Drive
Repositorio Git con CI obligatoria para mergear	GitHub Actions con 6 <i>jobs</i> (analyze, test, build-web, build-apk, semgrep, gitleaks)
Gestor de contraseñas para <i>secrets</i>	Bitwarden con bóveda dedicada al TFG; ningún <i>secret</i> commiteado
Copia del <i>keystore</i> Android	Tres ubicaciones independientes (USB cifrado, <i>bucket</i> privado, copia impresa)
Agenda con tutorías y entregas parciales	Calendario sincronizado con recordatorios 72 h antes de cada hito
Dispositivos físicos para NFC y <i>push</i>	Móvil Android personal con NFC; segundo dispositivo prestado para pruebas multi-cuenta
Lista de contactos para usuarios de prueba	Cinco perfiles confirmados por escrito al menos dos semanas antes de la sesión
Conectividad fiable	Conexión doméstica de respaldo (compartición desde móvil) y <i>coworking</i> académico como plan B

8. Entregables parciales

Entrega	Semana	Fecha	Contenido
ENTREGA 1	5	2026-05-05	Documentos <code>01_analisis.md</code> , <code>02_diseno_proyecto.md</code> , <code>03_planificacion.md</code> , repositorio Git público con estructura inicial del proyecto y diagrama de Gantt validado

Entrega	Semana	Fecha	Contenido
ENTREGA 2	9	2026-06-02	Aplicación funcional, suite de pruebas en verde, CI en verde, APK <i>release</i> firmado
ENTREGA FINAL	10	2026-06-09	Memoria completa (8 documentos TFG), manuales técnico y de usuario, presentación, repositorio limpio
DEFENSA	11	semana del 09-16/06/2026	Presentación oral ante tribunal y demostración en vivo

Cada entrega exige verificación previa: `flutter analyze` sin *issues*, `flutter test` íntegramente verde y CI sin fallos en el *commit* que se entrega.

9. Procedimiento de cierre de sprint

Cada sprint semanal se cierra el domingo con la siguiente lista de verificación:

1. `flutter analyze` con cero *issues*.
2. `flutter test` con cero fallos sobre los 679 tests del repositorio en el momento de la verificación.
3. `git status` con árbol limpio.
4. *Commit* convencional (`feat` , `fix` , `docs` , etc.) y *push* a `master` mediante PR de auto-revisión.
5. CI completa en verde (seis *jobs*: *analyze*, *test*, *build-web*, *build-apk*, *semgrep*, *gitleaks*).
6. Actualización de `docs/00_MAESTRO.md` y, si procede, del plan mega refinamiento.
7. Anotación retrospectiva si toca cierre de sprint número par.

Si alguno de los criterios falla, el sprint no se cierra y se decide si extender una jornada o trasladar el ítem al siguiente sprint con penalización documentada.

10. Resumen ejecutivo

El plan de Transitly cubre 11 semanas naturales del 1 de abril al 16 de junio de 2026, con un calendario coherente con un TFG de DAM y con un único autor a tiempo prácticamente completo. La metodología combina cascada para las fases que requieren documentación previa estable (análisis y diseño) con sprints semanales tipo Scrum solo para el desarrollo. El coste directo es cero; los riesgos críticos están identificados, mitigados y con contingencia escrita. Los entregables parciales se anclan a fechas verificables y a criterios de aceptación objetivos (CI verde, tests sin fallos, documentación sincronizada).

Adenda — Avance entre el 23 de mayo y el 4 de junio de 2026

Entre la firma del anchor original `b908f3c` (23/05) y el cierre intermedio `5231f4c` (04/06) se han ejecutado **94 commits** repartidos en cinco oleadas de estabilización derivadas del feedback recogido en el primer ciclo de pruebas. La planificación inicial no contemplaba con detalle esta fase intensiva de bugfix posterior al **hito de fin de desarrollo** (02/06), pero se ha integrado en el calendario sin tensionar el cierre del 16/06 gracias al margen reservado a la **semana 10** (seguimiento y evaluación).

Asignación temporal real entre 23/05 y 04/06:

Lote de trabajo	Periodo	Resultado
Reparación V3-V5 (personalización, fondos, paletas, dislexia, splash)	23-25 mayo	Bug crítico de fuentes neutralizado; recuperación de personalización persistente
Mega oleada 14 bugs (auth, widgets, theming, mapa, comunidad)	26-27 mayo	Cierre de 14 ítems con cobertura cross-sectorial
Crash ally + recovery boot	28-29 mayo	<code>BootCanary</code> + <code>RecoveryScreen</code> operativos
Plan 21 bugs (mapa, perfil, auth, comunidad)	30 mayo - 1 junio	16/21 cerrados en el primer pase; 5 cerrados en la oleada siguiente
Plan 8 mejoras + cierre auth Google + RLS 42P17 + release v1.11.0	2 - 4 junio	Release pública publicada en GitHub Releases con APK firmado

Impacto en el cronograma global: las semanas 6 a 9 (desarrollo) se cierran con un excedente funcional positivo (widgets configurables, wizard de rutas con tap en mapa, recovery boot) que no estaba en la lista mínima del MVP, y con cero deudas técnicas críticas bloqueantes. La **semana 10** (seguimiento y evaluación, 3-9 junio) absorbe la estabilización post-MVP sin desplazar la fecha de defensa.

Decisión documentada de alcance: se confirma sin cambios la exclusión de **GTFS-Realtime real** (pendiente del operador), **widget iOS completo**, **expansión a otras ciudades** y **modelo ML de ETA**, ya recogida en `01_analisis_contexto.md` y en `docs/EXTERNAL_BLOCKERS.md`. Adicionalmente se documenta como **deuda explícita:** configuración SMTP propia (la verificación de email queda bypassada durante el TFG con el flujo descrito en `docs/SUPABASE_SETUP.md`).

Riesgo materializado y resuelto: durante las pruebas con dispositivos físicos se detectó un crash nativo del engine al combinar opciones de accesibilidad. Inicialmente requería `adb shell pm clear` (borrado total). La mitigación implementada (boot canary + persistencia diferida + recovery UI) elimina el riesgo de aplicación brickeada y mantiene la usabilidad sin intervención del usuario más allá de un eventual botón "Restaurar configuración por defecto" en la pantalla de recovery.

Adenda 2 — Cierre del cronograma a 8 de junio de 2026

La **ENTREGA FINAL** (semana 10, 09/06/2026) se alcanza en plazo. Durante las semanas 10 y 11 se ejecutaron, dentro del margen planificado, las funcionalidades que cerraban objetivos parciales y la publicación de la documentación, sin desplazar la fecha de defensa:

Lote de trabajo	Periodo	Resultado
Push FCM real (cliente) + verificación en dispositivo	7-8 junio	App recibe push con la app cerrada; token en <code>device_tokens</code>
Modo conductor en segundo plano (foreground service)	7-8 junio	Seguimiento GPS continúa con pantalla bloqueada
Planificador origen→destino reactivado	7-8 junio	Flujo de trayectos con transbordos (<code>/route-plan</code>)
Web del proyecto en GitHub Pages + release v1.12.1	8 junio	Presentación, entregables y descarga del APK públicos

Cronograma cumplido sin desviación de la fecha de defensa. El coste directo se mantiene en 0 €. La planificación inicial (11 semanas, metodología híbrida cascada + Scrum solo) demostró robustez: absorbió tanto la estabilización post-MVP como las funcionalidades finales sin tensionar el hito de cierre.

04 — Desarrollo e Implementación

Proyecto: Transitly **Repositorio:** nexto-stop-v2 (anclaje original `master @ b908f3c` · 2026-05-23; estado actualizado `master @ b47180d0` · 2026-06-08, release v1.12.1) **Pila tecnológica principal:** Flutter 3.x · Dart 3 · Riverpod 2.6 · GoRouter 17 · Freezed 3 · Supabase (PostgreSQL + Auth + Edge Functions) · Hive 2.2 con cifrado AES en cajas sensibles · Firebase Messaging 16 · Sentry 8 · PostHog 5 ·

`flutter_secure_storage` · `very_good_analysis` · `leak_tracker_flutter_testing` **Indicadores verificados a 23/05/2026 (anchor original):** 616 *tests*, 14 migraciones SQL, 27 *features*, 4 Edge Functions, 5 ADRs, 6 *runbooks*, 628 claves ARB, 4 *jobs* de CI, *scorecard* TFG 8,9/10 y producción 6,0/10. **Indicadores verificados a 08/06/2026 (defensa):** 679 *tests* (0 fallos), 51 migraciones SQL consecutivas, 27 *features* (446 ficheros `.dart`, ~94k LOC), 8 Edge Functions desplegadas, 30 modelos *freezed*, 38 *widgets* compartidos, 642 claves ARB (es; cobertura EN/AR), 6 *jobs* de CI en verde, **release pública v1.12.1** (APK universal, Android 7.0+) en GitHub Releases (<https://github.com/astralk9999/Transitly/releases/tag/v1.12.1>). Ver §11 (evolución 23/05–04/06) y §13 (cierre a 08/06) al final del documento.

1. Metodología real aplicada

La metodología que ha guiado la construcción del producto se asienta sobre tres pilares: un enfoque **feature-first**, una variante de **Scrum solo** con sprints semanales y un compromiso estricto con la **trazabilidad** mediante *Conventional Commits* y CI bloqueante.

1.1. Feature-first y sprints semanales

El árbol `lib/features/` agrupa 27 *features* autocontenidas, cada una con su pantalla, sus *providers* Riverpod y, cuando procede, sus *widgets* internos. Las dependencias entre *features* se modelan a través de la capa `lib/data/`, lo que evita acoplamientos transversales y permite que cada sprint semanal se dedique a una o dos *features* nuevas, una integración o una fase de pulido. La planificación de cada sprint se realiza los lunes y el cierre el domingo, con verificación explícita (analyze, test, CI verde) antes de pasar al siguiente.

1.2. Conventional Commits y release-please

Todos los *commits* siguen el estándar *Conventional Commits*. Los prefijos en uso son `feat`, `fix`, `docs`, `test`, `chore`, `refactor`, `perf`, `ci`, `build` y `revert`. Esta disciplina permite a *release-please* generar automáticamente el `CHANGELOG.md` y proponer versiones SemVer. Los mensajes están redactados en imperativo y describen el "porqué" cuando no es obvio.

1.3. Code review propia mediante PRs auto-revisión

El proyecto es individual, pero todo cambio de cierto calado se introduce mediante *pull request*. La auto-revisión tiene un propósito auditable: forzar que CI ejecute las verificaciones automáticas

(analyze, test, build-web, build-android, gitleaks, semgrep) antes de incorporar el cambio a `master`. Esa disciplina ha permitido detectar regresiones y mantener los 4 *jobs* en verde de forma sostenida.

1.4. Asistencia de IA documentada

El proyecto ha utilizado asistencia de IA para redacción, generación inicial de *boilerplate* y revisión cruzada. La declaración es explícita y se hace por integridad académica. Cada *commit* lleva la huella del autor, los *prompts* relevantes están archivados y el contenido se revisa, atribuye y firma como propio.

2. Arquitectura implementada

Las decisiones de fondo están registradas en cinco **Architecture Decision Records (ADR)**:

ADR	Decisión	Motivo principal
001	Riverpod 2.6 como gestor de estado	Tipado fuerte, sin código generado obligatorio, <i>autoDispose</i> y <code>.family</code> idóneos para una app de movilidad con sesiones cortas
002	Freezed 3 para <i>value objects</i>	Inmutabilidad, igualdad estructural y <code>copyWith</code> sin <i>boilerplate</i>
003	Hive 2.2 para almacenamiento local	Velocidad, soporte de cifrado AES vía <code>HiveAesCipher</code> , ausencia de SQL en cliente
004	Supabase como <i>backend</i>	PostgreSQL con RLS nativo, Auth integrada, Edge Functions en Deno, <i>free tier</i> viable
005	Feature-first como organización	Escala mejor que <i>layer-first</i> cuando el equipo es pequeño y las <i>features</i> son verticales

2.1. Capas

lib/	
main.dart	Bootstrap secuencial
app.dart	MaterialApp.router + tema + locale
core/	Tokens, router, utils, theme
data/	Repositorios + cache + sync (12 dominios)
features/	Feature-first (27 features)
l10n/	ARB + generated (es, en, ar) – 642 claves
shared/	Modelos, providers y widgets reutilizables

2.2. Patrón canónico de repositorio con SWR

Cada dominio sigue el patrón **domain / local / remote / mock**:

```
lib/data/<entity>/
  domain/<entity>_repository.dart      interfaz abstracta
  remote/<entity>_remote_repository.dart cliente Supabase
  local/<entity>_local_repository.dart Hive
  local/<entity>_mock_repository.dart  modo invitado
  <entity>_repository_provider.dart    Riverpod SWR + selector
```

De los 12 dominios cableados, **4 funcionan sin SWR** por motivos documentados: `auth` (la sesión la gestiona el SDK de Supabase), `analytics`, `privacy_consent` y `nfc` (no requieren cache porque su estado lo manda el dispositivo o el cliente *backend* en tiempo constante).

2.3. Riverpod y *autoDispose*

Riverpod 2.6 expone `Provider`, `FutureProvider`, `StreamProvider`, `StateProvider` y `NotifierProvider`. Los providers que mantienen suscripciones (canales *realtime*, *streams* de notificaciones) usan `autoDispose` para cerrar recursos cuando ningún consumidor está escuchando. Los providers parametrizables emplean `.family` para evitar instanciaciones duplicadas.

2.4. Navegación con GoRouter 17

GoRouter aporta navegación tipada, soporte declarativo de *deeplinks* y *shell routes* para el chasis principal (rail/bottom según anchura). Los *guards* de autenticación viven en `redirect_guards.dart` y consultan el rol REAL del perfil en Supabase, no un estado mutable local.

3. Integración de bases de datos

3.1. Supabase (PostgreSQL)

El *backend* descansa en una instancia Supabase con PostgreSQL como motor relacional. La política RLS es **DENY-by-default**: ninguna tabla con datos sensibles concede acceso sin una *policy* explícita. La seguridad se apoya en dos helpers `SECURITY DEFINER`, `is_admin(uuid)` e `is_moderator_or_admin(uuid)`, con `search_path` fijado para evitar suplantación. Cada *policy* lleva su `COMMENT ON POLICY` con el caso de uso que la justifica, y existe una **separación estricta entre `service_role` y `anon`** para evitar escaladas.

Indicador	Valor
Migraciones SQL versionadas	51 consecutivas (núcleo + líneas/horarios COMUJESA, conductor en vivo, triggers de push, RLS helpers)
Tablas en <code>public</code>	núcleo de 16, ampliado con líneas/horarios, <code>driver_live_trips</code> , <code>device_tokens</code> , zonas y comunidad
Helpers <code>SECURITY DEFINER</code>	<code>is_admin</code> , <code>is_moderator_or_admin</code> , <code>is_route_owner</code> , <code>get_supabase_service_key</code> , <code>get_supabase_functions_url</code>
Política por defecto	DENY-by-default
Separación de claves	<code>service_role</code> frente a <code>anon</code>

3.2. Hive local

Hive 2.2 actúa como **cache offline-first**. En total se utilizan 16 *boxes*, de las cuales tres almacenan información sensible y se cifran con `HiveAesCipher` :

Box	Cifrado	Contenido
<code>authSessionMeta</code>	Sí (AES)	Metadatos de sesión auth
<code>userPreferences</code>	Sí (AES)	Preferencias personales
<code>pendingActions</code>	Sí (AES)	Cola offline de acciones del usuario
<code>routes</code> , <code>stops</code> , <code>schedules</code> , ...	No	Datos públicos del operador (rendimiento prioritario)

La clave de cifrado se conserva en `flutter_secure_storage` y nunca abandona el dispositivo.

4. Integración multimedia

Recurso	Tecnología	Notas
Caché de <i>tiles</i> del mapa	<code>flutter_map_tile_caching</code> v10	Política LRU con evicción configurable; mapa usable sin conexión
Tipografías empaquetadas	DM Sans (variable) + IBM Plex Mono en tres pesos (400, 500, 700)	<i>Assets</i> locales para evitar dependencias en red y descargas remotas
Logotipo y marca	PNG en <code>assets/branding/</code>	Distribuido junto al APK
<i>Shaders</i>	GLSL custom (<code>shaders/smoke.frag</code>)	Fondos animados sin coste de imagen

La estrategia general es **empaquetar todo lo que se pueda** para que la app arranque y funcione sin red, salvo la consulta al *backend* y las descargas opcionales de *tiles* nuevos.

5. Integración de interfaces

5.1. Sistema de diseño propio

El sistema reside íntegramente en `core/theme/` y se compone de cuatro fuentes únicas de verdad:

- `transit_colors` : paleta semántica (no colores crudos).
- `transit_typography` : escalas tipográficas.
- `transit_spacing` : rejilla de 4 puntos.
- `transit_animations` : duraciones y curvas reutilizables.

Está prohibido duplicar *tokens*; los *widgets* compartidos los consumen y nunca los re-implementan.

5.2. Tema accesible

El tema incorpora ocho matrices de **simulación de daltonismo** (deuteranopia, protanopia, tritanopia, acromatopsia y variantes parciales), un modo de **alto contraste** y un escalado tipográfico compuesto: se respeta el factor del sistema operativo mediante `MediaQuery.textScalerOf` y se *clampa* internamente entre 0,8 y 2,5 para evitar que la interfaz se rompa con valores extremos. Los *widgets* compartidos están auditados con *tests* de semántica.

5.3. Responsive scaffold

El chasis principal adapta su patrón de navegación al ancho de la pantalla: *bottom navigation bar* en móviles, *navigation rail* en tabletas y escritorio. La decisión es declarativa según *breakpoints* publicados en `core/theme/`.

5.4. Localización

`flutter_localizations` con tres locales activos (español como fuente, inglés y árabe). El recuento actual es de **642 claves ARB** en español, con cobertura en inglés y árabe. La dirección RTL está soportada para árabe.

6. Integración de servicios externos

6.1. Edge Functions Deno

Hay **ocho Edge Functions desplegadas** en Supabase, todas escritas en Deno y con `verify_jwt` habilitado cuando la naturaleza del *endpoint* lo permite:

Función	Propósito	Notas
<code>send_notification</code>	Envío <i>push</i> a través de FCM HTTP v1	OAuth JWT firmado en Deno; <i>fail-closed</i> si falla la persistencia; limpia tokens inválidos
<code>import_gtfs</code>	Importación de feeds GTFS	Anti-SSRF: resolución DNS estricta, redirecciones manuales, rangos privados bloqueados
<code>delete_user</code>	Borrado total (Art. 17 GDPR)	Limpia perfil, datos derivados y referencias
<code>purge_old_data</code>	Minimización (Art. 5 GDPR) y purga de datos antiguos	Ejecutada por <code>cron</code> ; idempotente
<code>generate_data_export</code>	Portabilidad (Art. 20 GDPR)	Exporta los datos del usuario en formato estructurado
<code>approve_user_route</code>	Moderación de rutas sugeridas	Promueve una ruta comunitaria tras revisión
<code>promote_stop_to_official</code>	Moderación de paradas	Convierte una parada comunitaria en oficial

Función	Propósito	Notas
<code>validate_share_code</code>	Códigos de invitación (conductor/operador)	Valida y canjea el código contra el rol correspondiente

6.2. Firebase Messaging

`firebase_messaging` 16 gestiona los *tokens* del dispositivo y la recepción de mensajes. La integración quedó **completa y verificada en v1.12.1**: el proyecto Firebase `transitly-ee8cf` (con su `google-services.json` y el plugin Gradle `com.google.gms.google-services`) inicializa al arranque, se registran los manejadores de mensajes en primer plano (`setupForegroundHandler`) y en segundo plano (`onBackgroundMessage`), y el *token* del dispositivo se persiste en la tabla `device_tokens` al iniciar sesión (y se elimina al cerrarla). En Android se declara el canal `transitly_push` en el manifiesto. Los *deeplinks* incluidos en el *payload* se consumen tanto en *foreground* como en *background* y en estado *killed*. La recepción con la app cerrada se verificó en dispositivo físico (`FlutterFirebaseMessagingBackgroundService started`). El envío programático desde el *backend* depende de la *service account* del operador, documentada como dependencia externa en `docs/FCM_SETUP.md`.

6.3. Observabilidad: Sentry y PostHog

Sentry 8 instrumenta **seis transacciones** representativas:

1. `auth.signIn`
2. `auth.refresh`
3. `map.initial_render`
4. `nfc.read`
5. `network.fetch_routes`
6. `push.send`

El *hook* `beforeSend` *scrubba* sistemáticamente datos personales identificables: correos electrónicos, *tokens* `Bearer`, *query parameters* sensibles y cabeceras `Authorization`.

PostHog 5 registra **17 eventos** funcionales (entre ellos `signup_completed`, `route_viewed`, `incident_reported` y los correspondientes a las pantallas críticas). El *autocapture* está **desactivado** por defecto y el opt-out es la posición inicial; toda captura requiere consentimiento explícito (*consent-gated*).

6.4. Almacenamiento seguro

`flutter_secure_storage` conserva las claves AES de Hive y los *tokens* ofuscados que requieran persistir entre arranques.

7. Pruebas técnicas

La suite actual cuenta con **679 tests pasando** (0 fallos). Esta cifra se verifica a fecha de defensa (`master @ b47180d0`). El desglose aproximado por categoría es el siguiente:

Categoría	Cantidad aproximada	Foco
Unitarios	≈ 340	Modelos Freezed, <i>helpers</i> , <i>providers</i> Riverpod, <i>theme</i> , utilidades
Widget	≈ 220	Componentes compartidos, pantallas críticas, semántica
Integración	3 <i>happy paths</i> en <code>integration_test/</code>	Registro/inicio, búsqueda, reporte de incidencia
Repositorios remotos	12 dominios	<code>SupabaseClient</code> mockeado
Leak tracking	Transversal	<code>leak_tracker_flutter_testing</code> para controladores y <i>streams</i>

Visual goldens: los ocho *widgets* compartidos y dos brillos (claro/oscuro) cuentan actualmente con *rendering tests*; la evolución hacia *goldens* pixel-perfect está planificada y registrada como evolución, no como deuda crítica.

`mocktail` cubre la inyección de mocks sin generación de código. Para pruebas con tiempos se utiliza `fake_async`. La regla `avoid_print` está activa en `lib/`; solo `AppLogger` produce salida.

8. Pruebas funcionales

8.1. Accesibilidad manual

Está prevista para la semana 10 una pasada manual de **accesibilidad con TalkBack (Android) y VoiceOver (iOS)**. El procedimiento, las incidencias y el resultado quedan registrados en `docs/A11Y_MANUAL_TEST_2026_06.md`. La pasada cubre orden de foco, etiquetas semánticas, contraste y operación con una sola mano.

8.2. Push extremo a extremo

Se realiza una prueba E2E del flujo *push* cubriendo los tres estados del proceso receptor (*foreground*, *background* y *killed*) y el seguimiento del *deeplink* incluido en el *payload*. La verificación se hace en un dispositivo físico Android porque los emuladores no emulan fielmente la entrega FCM en *killed*.

8.3. Sesión con usuarios reales

En la semana 10 se ejecuta una sesión con **cinco usuarios reales** de los perfiles definidos en el documento 03, seguida de un cuestionario **SUS (System Usability Scale)**. Los resultados se incorporan al documento 05 (Evaluación y documentación) con análisis cuantitativo y cualitativo.

9. Documentación del código y del proyecto

9.1. `dartdoc` y publicación

La API pública se documenta con `dartdoc`. El sitio resultante se publica en **GitHub Pages**, lo que permite consultar la documentación generada sin clonar el repositorio.

9.2. Documentos vivos

Documento	Propósito
<code>README.md</code> raíz	Visión general, badges de CI, <i>codecov</i> , licencia y versión
<code>AGENTS.md</code>	Guía para agentes de IA colaboradores; recoge reglas de arquitectura no negociables
<code>CHANGELOG.md</code>	Generado por <code>release-please</code> a partir de los <i>Conventional Commits</i>
<code>docs/00_MAESTRO.md</code>	Cuadro de mando central del proyecto
<code>docs/adr/</code>	Cinco ADRs (Riverpod, Freezed, Hive, Supabase, feature-first)
<code>docs/runbooks/</code>	Seis <i>runbooks</i> operativos (despliegue, rotación de claves, recuperación de Supabase, <i>keystore</i> Android, <i>push</i> en producción, GDPR)
<code>docs/tfg/</code>	Memoria académica (ocho documentos)
Resto de <code>docs/</code>	Más de 70 documentos técnicos (arquitectura, escalabilidad, accesibilidad, seguridad, <i>performance budget</i> , etc.)

9.3. Estilo de comentarios

Como regla general, el código no se comenta. Los identificadores expresan el "qué". Solo se comenta el "porqué" no obvio: invariantes ocultos, *workarounds* con referencia al *bug* o decisiones contraintuitivas que ahorrarían horas a una persona que lea el código por primera vez.

10. CI/CD

10.1. *Jobs* actualmente en verde

GitHub Actions ejecuta los siguientes *jobs* en cada *push* y *pull request* contra `master`:

1. **Flutter Analyze:** `flutter analyze` con cero *issues* obligatorio.
2. **Flutter Test:** `flutter test --coverage` con umbral mínimo de cobertura y *upload* a Codecov.
3. **Build Web (release):** compilación en *release* validando que el código de la rama compila para plataforma web.
4. **Build Android APK:** compilación *release* con `--split-per-abi`, `--obfuscate` y `--split-debug-info`; firma con claves de *debug* en CI (la firma con *keystore* real ocurre en *release* manualmente). Incluye verificación de **presupuesto de tamaño** por ABI.
5. **Gitleaks:** escaneo de *secrets* con configuración `.gitleaks.toml`.

6. **Semgrep**: SAST con reglas en `.semgrep/rules.yaml`.

Los seis *jobs* (analyze, test, build-web, build-apk, semgrep, gitleaks) están **en verde de forma sostenida**. Gitleaks actúa como **bloqueante**: cualquier *secret* detectado impide el *merge*.

10.2. Evolución prevista

Está planificado añadir un *job* dedicado a *tests* de Edge Functions, alcanzando un total de siete *jobs*. Esa evolución no condiciona la entrega académica y forma parte del plan mega refinamiento.

10.3. Hooks locales con lefthook

El proyecto utiliza **lefthook** para ejecutar verificaciones antes de cada *commit* y antes de cada *push*. Los *hooks* incluyen:

- `flutter analyze` (cero *issues*).
- `dart format --set-exit-if-changed`.
- *Check* de coherencia del estado del repositorio (`verify-state`).
- Gitleaks local antes de *push*.

10.4. Releases

Los *releases* siguen versión SemVer mediante etiqueta Git que dispara `release-please`. El *workflow* genera automáticamente el *release* en GitHub a partir del `CHANGELOG.md`.

11. Decisiones técnicas relevantes

Las cinco decisiones de mayor calado están explicadas en sus ADRs y se resumen aquí porque condicionan el resto del desarrollo:

ADR 001 — Riverpod 2.6 como gestor de estado. Se valoraron Bloc y Provider clásico. Riverpod gana por dos motivos: tipado fuerte sin código generado obligatorio y soporte de `autoDispose` y `.family`, ideales para una app con pantallas que aparecen y desaparecen y con *streams* en tiempo real.

ADR 002 — Freezed 3 para inmutabilidad. El coste de añadir un generador se compensa por la ausencia de errores comunes con `copyWith` manuales, igualdad estructural correcta y patrones de *sealed classes* para *states*.

ADR 003 — Hive 2.2 con HiveAesCipher. Hive ofrece persistencia local muy rápida sin SQL embebido, suficiente para una app móvil con datasets modestos. El cifrado AES nativo permite cubrir las tres *boxes* sensibles sin librerías adicionales.

ADR 004 — Supabase como backend. Combina PostgreSQL con RLS nativo (justo lo que requiere un proyecto con datos personales bajo GDPR), Auth integrada, Edge Functions en Deno y un *free tier* viable para un TFG.

ADR 005 — Feature-first. La organización por *features* verticales escala mejor que la organización por capas cuando el equipo es pequeño y cada *feature* tiene su pantalla, su *provider* y, eventualmente, sus *widgets* internos.

12. Resumen ejecutivo

El producto entregado es **funcional, verificable y trazable**. Cada *commit* atraviesa `flutter analyze` con cero *issues*, los 679 *tests* y seis *jobs* de CI sostenidamente en verde. El *backend* descansa sobre PostgreSQL con RLS DENY-by-default y cincuenta y una migraciones versionadas; el cliente cifra los datos sensibles con AES en tres *boxes* de Hive y nunca expone *secrets* al repositorio gracias a Gitleaks. La observabilidad cubre seis transacciones y 17 eventos funcionales con *scrubbing* sistemático de PII y *consent gating* por defecto. Las decisiones de arquitectura están documentadas en cinco ADRs y las operaciones críticas en seis *runbooks*. La asistencia de IA queda declarada con transparencia. El siguiente documento, `05_evaluacion_documentacion.md`, presenta los procedimientos de evaluación, los resultados de la sesión con usuarios reales y los indicadores finales del proyecto.

11. Evolución entre 23/05 y 04/06 de 2026

Tras el cierre del anchor original `b908f3c`, la fase de estabilización post-MVP ha incorporado **94 commits** organizados en cinco oleadas de trabajo. Esta adenda documenta las features añadidas, las mitigaciones de bugs nativos y los cambios SQL relevantes.

11.1. Nuevas features añadidas

Widgets Android configurables (`lib/features/widgets_config/`). Tres pantallas

(`WidgetsConfigScreen`, `WidgetNextBusConfigScreen`, `WidgetMyLineConfigScreen`, `WidgetNfcBalanceConfigScreen`) permiten al usuario configurar cada uno de los tres widgets de pantalla de inicio Android desde el perfil. Cada pantalla incluye preview visual del widget renderizado con datos reales, formulario con buscador `RouteAutocomplete` reutilizable, dedupe de paradas vía extensión `MockDataServiceExt.getUniqueStopsForRoute` y botón "Probar widget" que escribe a `SharedPreferences` y llama `HomeWidget.updateWidget()`. Las cadenas se localizan en `widgetsConfig*` (ES/EN/AR).

Wizard de creación de rutas con tap en mapa

(`lib/features/create_route/widgets/map_stop_picker_screen.dart`). Sustituye el formulario manual de coordenadas por una pantalla con `FlutterMap` y handler `onTap` que coloca un pin en el punto seleccionado. Integra `mapSearchResultsProvider` (Nominatim/OSM) para buscar lugares por nombre. Incluye selector horizontal de tipo de parada con iconos (hotel, gasolinera, etc.) y devuelve un `WizardStop` al wizard principal vía `Navigator.pop`.

Filtros del mapa con árbol jerárquico (`lib/features/map/widgets/zone_company_line_tree.dart`).

Sustituye los chips planos por un árbol de tres niveles (zona → operador → líneas) con `ExpansionTile` anidado y checkbox tri-state (todos seleccionados, ninguno, mixed). El estado `MapFilterState` añade el set `disabledRouteIds` para filtrar a nivel de línea individual.

Boot canary y recovery screen (`lib/core/utils/boot_canary.dart`,

`lib/features/recovery/recovery_screen.dart`). Sistema de detección de crashes nativos en arranque mediante un flag en `SharedPreferences` que se marca como `BOOTING` al inicio de `main()` y se sustituye

por `STABLE` tras el primer `addPostFrameCallback`. Si el flag queda en `BOOTING` al siguiente arranque (la app crasheó antes del primer frame), se incrementa un contador de crashes consecutivos y se revierte la última preferencia sensible (`dyslexiaFontEnabled`, `highContrast`, `fontScale`, `colorBlindMode`, `backgroundId`). Tras dos crashes consecutivos se monta `RecoveryScreen` con `MaterialApp` propio sin shaders ni fuentes custom, ofreciendo botones para restaurar valores por defecto o continuar sin cambios.

TilePrewarmer idempotente (`lib/data/fmtc/tile_prewarmer.dart`). Pre-descarga al primer arranque los tiles del bbox de Jerez (zoom 13-15) usando FMTC v10. Verifica `store.stats.all.length >= 50` para saltar la descarga si el store ya está hidratado; permite uso offline en arranques posteriores sin red.

11.2. Estabilización del flujo de autenticación

Bypass de verificación de email. El listener `onAuthStateChanged` en `auth_repository_supabase.dart` salta el check `emailConfirmedAt` y emite `AuthAuthenticated` directamente. En `signUpWithEmail` se añadió un auto-login defensivo: si `signUp` retorna `User` pero `currentSession` es null (caso de tener "Confirm email" ON en Supabase Dashboard), se hace `signInWithPassword` con las credenciales recién creadas para garantizar sesión activa. Documentado en `docs/SUPABASE_SETUP.md`.

Google Sign-In: replay stream + navegación directa. Se identificó que `StreamController.broadcast` no replicaba el último estado a nuevos suscriptores de Riverpod, dejando al `signin_screen` en `AuthLoading` tras login exitoso con Google. La solución combina dos mecanismos: (a) el getter `authState` ahora es un `async*` que primero `yield`-a `_lastState` y luego delega al stream del controller, garantizando que cualquier suscripción nueva reciba el estado actual; (b) el botón Google en la UI navega directamente con `context.go('/home/inicio')` tras `await signInWithGoogle()` sin depender del listener del provider. Todos los `_stateController.add(...)` se reemplazaron por `_emit(...)` que actualiza `_lastState` antes de notificar.

11.3. Migración SQL y RLS

`fix_route_shares_rls_recursion`. PostgreSQL detectaba un ciclo entre `route_shares.route_shares_select_owner` y `routes.routes_select_visible` (ambas hacían `EXISTS (SELECT FROM <otra_tabla>)`) que devolvía error **42P17** ("infinite recursion in policy"). Se creó la función `public.is_route_owner(p_route_id uuid)` con `SECURITY DEFINER STABLE` que ejecuta el lookup `SELECT EXISTS FROM routes WHERE owner_id = auth.uid()` bypassando RLS, lo que rompe el ciclo. Las tres políticas de `route_shares` que consultaban `routes` (SELECT, DELETE, INSERT del owner) llaman ahora a la función. La semántica de visibilidad se preserva intacta.

11.4. Theming y accesibilidad

Fondo cross-tab. Los `Scaffold` de `home_shell.dart` y `map_tab.dart` se cambiaron a `Colors.transparent` para que el `BackgroundWrapper` envolvente fuera visible. Anteriormente sólo se aplicaba en la pantalla "Apariencia" porque los demás Scaffolds tapaban el wrapper con `c.bgRoot`.

Paleta en modo claro. Las 5 paletas no-default (Sunrise, Forest, Midnight, Ocean, Mono) dejaron de declarar `lightScheme: TransitLightColors()`. Al hacerlo, el provider cae al fallback `_deriveLightFromDark(palette.darkScheme)` que invierte las luminosidades HSL y mantiene el matiz característico de cada paleta (naranja en Sunset, verde en Forest, etc.) también en modo claro.

HighContrastSchemeWrapper. Sustituye al `HighContrastTheme.apply` previo. Implementa la interfaz `TransitColorScheme` y fuerza texto/fondo a valores puros (`#000000` / `#FFFFFF`) y accent a alto contraste (`#FFFF00` / `#0000FF`). Acepta un parámetro `preserveAccent` para mantener el accent de la paleta del usuario cuando se prefiere coherencia visual sobre el accent rígido de alto contraste.

11.5. Observabilidad en producción

AppLogger.warn/error sin guard `kDebugMode` . Anteriormente los métodos `warn` y `error` sólo emitían a `logcat` en debug builds, lo que impedía diagnosticar incidencias en APK release. Se eliminó el guard para `warn` y `error` (sólo afecta a esos dos niveles; `info` , `debug` y `perf` siguen siendo debug-only). El cambio permitió capturar el flujo de Google Sign-In en producción y diagnosticar el bug del stream descrito en §11.2.

11.6. Release pública v1.11.0

El APK release se publica como asset de release en GitHub

(<https://github.com/astralk9999/Transitly/releases/tag/v1.11.0>). Los 9 APKs históricos que se versionaban en `presentation/public/` se eliminaron del HEAD del repositorio (~792 MB liberados del `working tree`) y se añadió el patrón `*.apk` al `.gitignore` raíz. La presentación web (`presentation/src/components/Section01Hero.astro` , `Section14Download.astro` , `Layout.astro`) se modificó para enlazar a <https://github.com/astralk9999/Transitly/releases/latest> , una URL estable que apunta siempre a la última versión publicada.

13. Cierre a 8 de junio de 2026 (release v1.12.1)

La fase final, previa a la defensa, completó los objetivos funcionales que aún se cubrían de forma parcial y publicó la web del proyecto. Todo dentro del alcance original.

13.1. Notificaciones push reales (FCM) extremo a extremo

Se completó el **cliente** Firebase Cloud Messaging: integración de `google-services.json` (proyecto `transitly-ee8cf`), activación del plugin Gradle `com.google.gms.google-services` , claves reales en `firebase_options.dart` , registro de `onBackgroundMessage` y `setupForegroundHandler` en `main.dart` , y persistencia del `token` del dispositivo en `device_tokens` al iniciar sesión, con borrado al cerrar (`auth_repository_supabase.dart` → `_syncPushToken` / `_removePushToken` , ambos con guarda `kIsWeb`). El **pipeline servidor** —Edge Function `send_notification` (OAuth JWT → FCM HTTP v1, limpieza de tokens inválidos) y disparadores SQL `010_push_triggers.sql` — ya existía. Verificación en dispositivo físico: la app arranca sin fallos con FCM activo y recibe push con la app cerrada. La *service account* del operador para el envío programático queda documentada en `docs/FCM_SETUP.md` .

13.2. Modo conductor en segundo plano (foreground service)

El seguimiento GPS del conductor (`lib/features/driver/driver_live_screen.dart`) pasó de un `Timer.periodic` en primer plano a `Geolocator.getPositionStream` con un **foreground service** de Android (`AndroidSettings.foregroundNotificationConfig` , `enableWakeLock: true` , notificación persistente con icono `ic_notification`). La posición se sigue emitiendo al canal Realtime con la app

en segundo plano o la pantalla bloqueada; al terminar la ruta se cancela el `StreamSubscription` y la notificación. Permisos `FOREGROUND_SERVICE` , `FOREGROUND_SERVICE_LOCATION` y `WAKE_LOCK` añadidos al `AndroidManifest.xml` . En web se conserva el envío periódico mientras la pestaña está activa.

13.3. Planificador origen→destino reactivado

Se reactivó la pestaña **Buscar** (`lib/features/home/tabs/search_tab.dart`) y el flujo origen→destino con `RouteSearchBar` , navegando a `/route-plan` (`RoutePlannerService` : rutas directas y con un transbordo sobre el grafo de paradas).

13.4. Web del proyecto en GitHub Pages

Se corrigió el modo claro de la web (velo de contraste y blobs adaptables al tema) y se publicó el sitio en **GitHub Pages** con la presentación del proyecto, los entregables del TFG navegables y la descarga del APK resuelta automáticamente desde la release más reciente vía la API de GitHub. Release **v1.12.1: APK universal** (arm64-v8a, armeabi-v7a, x86_64), Android 7.0+, que sustituye al anterior APK por ABI única.

13.5. Evolución de métricas respecto al anchor original

Métrica	Anchor (23/05)	Defensa (08/06)
Tests	616	679
Migraciones SQL	14	51
Edge Functions	4	8
Claves ARB (es)	628	642
Jobs CI	4	6
Release pública	—	v1.12.1 (APK universal)

05 — Seguimiento, Evaluación y Documentación

Proyecto: Transitly (nexto-stop-v2) **Rama / HEAD original:** `master @ b908f3c` **Fecha de cierre del anchor original:** 2026-05-23 **Rama / HEAD actualizado:** `master @ b47180d0` (2026-06-08) **Release pública distribuida:** v1.12.1 (APK universal) — <https://github.com/astralk9999/Transitly/releases/tag/v1.12.1> **Ciclo formativo:** DAM (Desarrollo de Aplicaciones Multiplataforma) **Autoria:** Koldo Uruburu (autoría individual; asistencia IA documentada).

1. Procedimientos de control y seguimiento aplicados

El proyecto se ha desarrollado a lo largo de once semanas (01/04/2026 a 16/06/2026) y la disciplina de control se ha apoyado en una idea rectora: la documentación solo cuenta si es **verificable contra el código**. Toda cifra publicada en la memoria del TFG se genera o se contrasta con un script reproducible, y los cierres documentales se aceptan únicamente cuando existe un commit que los respalde.

1.1. Integración continua

Se mantiene un pipeline en GitHub Actions (`.github/workflows/ci.yml`) con **seis trabajos** ejecutados en cada push y en cada pull request hacia `master` :

1. **Flutter Analyze** — ejecuta `flutter analyze` con el preset `very_good_analysis` . Toda incidencia de tipo error, warning o info bloquea el avance.
2. **Flutter Test** — ejecuta `flutter test --coverage` , sube `coverage/lcov.info` como artefacto, válida el umbral global y comprueba budgets por módulo.
3. **Build Web (release)** — compila el target web para detectar regresiones de tree-shaking, tipos y l10n.
4. **Build Android APK** — construye el APK release y válida el budget de tamaño, verificando el flujo de release completo.
5. **Semgrep** — análisis estático de seguridad (bloqueante en findings de severidad ERROR).
6. **Gitleaks** — escaneo de secretos en el árbol y el historial.

Cuando el pipeline esta rojo, no se avanza en plan ni en documentación: la regla es **CI verde antes de cerrar nada**.

1.2. Hooks locales (lefthook)

El control en la máquina del desarrollador se ha automatizado con `lefthook` . Las reglas activas son:

- **pre-commit:** `flutter analyze` , `dart format --set-exit-if-changed` y `tool/verify_state.sh` . Garantiza que los datos publicados en `docs/00_MAESTRO.md` (número de tests, migraciones, features, ARB keys) coinciden con la realidad antes de aceptar el commit.
- **pre-push:** `flutter test` y `flutter analyze` . Evita publicar a remoto un estado que la CI tumbaria.

1.3. Auditorias independientes periodicas

Durante el ciclo se han ejecutado tres auditorias formales que actuan como contrapeso al sesgo de autor:

- **2026-04-15:** auditoria inicial sobre el alcance y la arquitectura base.
- **2026-05-17:** auditoria intermedia, documentada en `docs/historico/REVISION_INDEPENDIENTE_2026_05_17.md` , focalizada en deuda de capa de datos y observabilidad.
- **2026-05-22:** auditoria deep-dive con trece sub-agentes en paralelo, archivada en `docs/historico/AUDIT_2026_05_22.md` . Cubrio accesibilidad, RLS, contraste, observabilidad, dependencias, build Android y backend, y produjo la base del plan de remediacion v2.

1.4. Code review y plan vivo

Al tratarse de autoria individual, el code review se ha realizado mediante **self-review estructurado sobre pull requests internas**: cada cambio significativo se prepara como rama, se revisa contra los criterios de aceptacion del mega plan y se fusiona solo si los hooks pasan. El plan vivo (`docs/MEGA_PLAN_REFINAMIENTO.md`) clasifica cada ítem en P0-P3 y se actualiza tras cada cierre real.

El documento maestro (`docs/00_MAESTRO.md`) actua como fuente única de verdad: sus cifras se autogeneran con `tool/verify_state.sh` , que cuenta tests, migraciones, features, ARB keys y commits, y deja constancia del HEAD. Cualquier discrepancia entre lo afirmado y lo medido bloquea el cierre del cambio.

1.5. Plan de remediacion v2

Cuando la auditoria del 22 de mayo identifico desviaciones críticas, se redacto un plan de remediacion estructurado (`docs/historico/PLAN_ACCION_REMEDIACION_v2.md`) en seis fases ejecutadas en paralelo: capa de datos, observabilidad, accesibilidad, build Android, backend Supabase y documentación. Cada fase tiene criterios de aceptacion explicitos (grep verificable, commits asociados, tests anadidos) y un responsable IA por fase con supervisión humana.

2. Registro de incidencias

Las incidencias significativas detectadas durante el desarrollo se han registrado con fecha, descripcion, categoria, severidad y resolucion. Se listan a continuación las doce más representativas; el registro completo, junto con los SHA exactos de los commits que las cierran, vive en el historial de Git y en `docs/historico/` .

Fecha	Descripcion	Categoria	Severidad	Resolucion
2026-04-22	El APK release no compilaba: <code>workmanager</code> arrastraba la antigua v1 embedding incompatible con Flutter 3.x.	Build	Alta	Eliminacion del plugin de <code>pubspec.yaml</code> y rebajada de funcionalidad a un <code>Timer</code> controlado por <code>WidgetsBindingObserver</code> .
2026-04-29	Drift entre cifras de tests en cinco documentos (175 / 201 / 245 / 292 / 304).	Documental	Media	Creación de <code>tool/verify_state.sh</code> y consumo único en <code>00_MAESTRO.md</code> .
2026-05-04	La migración <code>014_audit_log.sql</code> duplicaba la tabla introducida en <code>001_init.sql</code> con un esquema incompatible.	Backend	Crítica	Renumeracion a <code>016_data_exports.sql</code> y conservacion de la auditoria como ampliacion (<code>011_audit_log_extras.sql</code>).
2026-05-08	Bug funcional <code>onPressed: () {}</code> en <code>route_detail_screen.dart</code> (línea 181) impedía marcar la ruta como favorita.	Funcional	Alta	Cableado a <code>userFavoritesProvider</code> y test unitario que cubre el callback.
2026-05-12	El color <code>textlo</code> fallaba el contraste WCAG AA: 2,2:1 en tema oscuro y 3,1:1 en tema claro.	Accesibilidad	Alta	Aumento de luminancia + script regenerable <code>dart run tool/contrast_check.dart</code> integrado en <code>verify_state.sh</code> .
2026-05-15	El proyecto Supabase quedo pausado por inactividad del plan gratuito.	Infraestructura	Media	Reactivacion manual del proyecto y rotacion de la <code>publishable key</code> documentada.
2026-05-18	El scaffold de <code>integration_test</code> estaba roto: el paquete no figuraba en <code>pubspec.yaml</code> .	Test	Media	Incorporacion de la dependencia y tres pruebas felices (boot, login mock, navegación principal).
2026-05-20	TalkBack no leía el <code>_NotificationBell</code> de la pantalla <code>home</code> .	Accesibilidad	Alta	Envolver el icono en <code>TransitIconButton</code> con <code>Semantics(label: ...)</code> localizado en ES/EN/AR.
2026-05-21	Cero spans en Sentry pese a que <code>docs/SENTRY_SPANS.md</code> describía seis transacciones objetivo.	Observabilidad	Alta	Instrumentacion de las seis transacciones (arranque, mapa, busqueda, login, escaneo NFC, sincronizacion).
2026-05-21	Cero eventos en PostHog pese a documentarse diecisiete en <code>docs/POSTHOG_EVENTS.md</code> .	Observabilidad	Alta	Cableado de los diecisiete eventos en las features correspondientes y test que comprueba el mock client.

Fecha	Descripcion	Categoria	Severidad	Resolucion
2026-05-22	<code>AppLogger</code> quedaba como noop en release, lo que hacia perder los breadcrumbs de Sentry.	Observabilidad	Media	Movimiento de <code>addBreadcrumb</code> fuera del bloque <code>kDebugMode</code> y test asociado.
2026-05-23	Factoria de repositorios duplicada en doce providers (código casi identico).	Deuda técnica	Media	Helper <code>repositoryWithSessionFallback</code> reutilizable y refactor coordinado de los doce providers.

El conjunto de incidencias se encuentra cerrado al cierre del anchor (HEAD `b908f3c`), salvo los diecinueve bloqueadores externos enumerados en `docs/EXTERNAL_BLOCKERS.md`, que dependen de servicios o decisiones fuera del alcance individual (alta en Play Console, certificado Apple, alta de COMUJESA como entidad colaboradora, etc.).

3. Cambios y mejoras documentados

La evolucion de Transitty se materializa en un historial de commits trazables con la convencion `feat:`, `fix:`, `refactor:`, `test:` y `docs:`. La tabla siguiente resume los grandes bloques de mejora; los SHA exactos pueden consultarse en `git log` y en `CHANGELOG.md`.

Bloque	Naturaleza	Descripcion	Cobertura tests antes / después
Estabilizacion del build Android	Fix	Eliminacion de <code>workmanager</code> , paso a Kotlin DSL puro, signing condicional, ABI splits, <code>--obfuscate</code> .	304 → 314
Saneamiento de migraciones SQL	Fix	Renumeracion y consolidacion en catorce migraciones consecutivas con RLS default-deny.	314 → 334
Refactor de modelos a <code>@freezed</code>	Refactor	Veintisiete modelos inmutables con <code>copyWith</code> , <code>==</code> , <code>hashCode</code> y serializacion <code>json_serializable</code> .	334 → 376
Cierre de PRO-A11Y completo	Feat	Contraste regenerable, <code>textScaler</code> correcto, semántica localizada en ES/EN/AR, foco visible, tap targets 48 dp.	376 → 500
Cierre de PRO-Rel (release readiness)	Feat	Pipeline de release con AAB firmado, budget de APK, debug-info archivado, splits ABI.	500 → 480 (regresion controlada y recuperacion)
Observabilidad real	Feat	Seis spans Sentry, diecisiete eventos PostHog, breadcrumbs en release, consent-gating efectivo.	500 → 523

Bloque	Naturaleza	Descripcion	Cobertura tests antes / después
Capa de datos con repositorio canonico	Refactor	Doce repositorios con domain/remote/local/mock/provider y SWR; helper de fallback de sesión.	523 → 570
Cierre de PRO-Ops y EXTERNAL_BLOCKERS	Docs + feat	Runbooks operativos, error budget policy, documentación de bloqueadores externos.	570 → 616

El detalle completo (commits + PRs internas) puede consultarse en `git log --oneline` desde el commit raíz hasta `b908f3c`, junto con los archivos `docs/MEGA_PLAN_REFINAMIENTO.md` y `docs/historico/AUDIT_2026_05_22.md`.

4. Feedback de usuarios

4.1. Sesión de validacion (semana 10)

El plan académico contempla una sesión de validacion en la semana 10 del cronograma (prevista para el 04/06/2026) con **cinco usuarios** de perfiles diversos: pasajero senior con baja experiencia digital, pasajero joven habituado a apps de transporte, usuario con visión reducida que utiliza lector de pantalla, conductor en activo y administrador municipal.

4.2. Tareas guiadas

Cada participante completa, en orden, las siguientes tareas, observadas por el autor sin intervenir:

1. Crear una cuenta nueva con email y contraseña.
2. Consultar la línea L1 y revisar la siguiente parada.
3. Reportar una incidencia ficticia desde el detalle de la ruta.
4. Cambiar el idioma de la aplicación a ingles y verificar el cambio.
5. Descargar una región offline (Centro de Jerez) y validar la persistencia.
6. Leer una tarjeta NFC simulada para comprobar saldo (mock controlado).

4.3. Instrumento de medicion

La medicion principal es el **System Usability Scale (SUS)** clasico, con sus diez ítems en escala Likert de cinco puntos. El objetivo del proyecto es alcanzar una puntuacion media $SUS \geq 75$ (umbral de "good" en la escala interpretativa de Bangor, Kortum y Miller). De forma complementaria se registra el tiempo de finalizacion por tarea, los errores observables y las preguntas verbales emitidas por el participante.

4.4. Hallazgos preliminares y acciones

A partir de los pilotos internos (autora, dos revisores académicos y un conductor voluntario), los hallazgos cualitativos que ya han generado cambio en código son:

- **Positivo:** claridad de los iconos del bottom navigation, paleta accesible para daltonismo (probada con simulador de Coblis), funcionamiento offline en ausencia de cobertura.
- **A mejorar:** latencia inicial al cargar el mapa con muchas paradas (mitigada con clustering basado en zoom); nomenclatura arabe imprecisa en algunos campos (corregida tras revisión lingüística de los 642 strings ARB); peso del APK (deuda reconocida; el universal de v1.12.1 prioriza compatibilidad sobre tamaño).

La sesión formal con los cinco usuarios externos producirá un anexo en

`docs/historico/SUS_2026_06_04.md` con los datos en bruto anonimizados y el cálculo agregado.

5. Métricas finales auditadas

Las cifras siguientes son las verificadas a fecha de defensa (`master @ b47180d0` , release v1.12.1) y se publican como contrato del cierre del TFG:

Métrica	Valor	Comentario
Tests automáticos en verde	679	Cobertura distribuida entre widgets, modelos, accesibilidad, utilidades y data layer (0 fallos).
Ficheros de código Dart	446 (~94k LOC)	Sin contar código generado (<code>.g.dart</code> , <code>.freezed.dart</code>).
Cobertura global de tests	24-30 %	Buena en componentes UI y modelos, débil en capa <code>data/remote</code> (deuda reconocida y priorizada en P2-4).
Migraciones SQL aplicadas	51 consecutivas	Schema completo con RLS default-deny, funciones helper, líneas/horarios COMUJESA, conductor en vivo y triggers de push.
Edge Functions desplegadas	8	<code>send_notification</code> , <code>import_gtfs</code> , <code>delete_user</code> , <code>purge_old_data</code> , <code>generate_data_export</code> , <code>approve_user_route</code> , <code>promote_stop_to_official</code> , <code>validate_share_code</code> .
Jobs CI en verde	6 / 6	Analyze, Test, Build Web, Build APK, Semgrep, Gitleaks.
Modelos <code>@freezed</code>	30	Inmutables, con <code>copyWith</code> , <code>==</code> , <code>hashCode</code> y serialización.
Widgets compartidos	38	Reutilizados en toda la app (<code>lib/shared/widgets/</code>).
Runbooks operativos	6	Disaster recovery, error budget, migration rollback, push down, Sentry spike, Supabase down.
Documentos vivos	73+	En <code>docs/</code> (incluye <code>tfg/</code> , <code>runbooks/</code> , <code>adr/</code> , <code>historico/</code>).
Claves ARB	642 en español (cobertura EN/AR)	Cobertura completa de la UI (incluye RTL en árabe).

Metrica	Valor	Comentario
Release pública	v1.12.1 (APK universal, Android 7.0+)	Distribuida en GitHub Releases; descarga resuelta desde la web.
Scorecard interno	TFG 8,9 / 10 — Produccion 6,0 / 10	Diferenciacion explicita entre madurez académica y madurez productiva.

6. Plan de elaboracion de manuales

La documentación entregable se ha estructurado en cuatro frentes complementarios:

1. **Manual técnico** ([docs/tfg/06_manual_tecnico.md](#)). Cubre requisitos, instalación, configuración avanzada, estructura del repositorio, despliegue de backend Supabase, configuración de CI/CD, mantenimiento rutinario, runbooks resumidos y resolucion de problemas habituales. Es el documento de referencia para cualquier desarrollador que herede el proyecto.
2. **Manual de usuario** ([docs/tfg/07_manual_usuario.md](#)). Cubre el uso de la aplicación desde la perspectiva del pasajero, conductor y administrador. Incluye capturas, flujos paso a paso y consejos de accesibilidad.
3. **Runbooks operativos** ([docs/runbooks/](#)). Seis documentos operativos pensados para incidentes reales en producción: recuperacion ante desastre, política de error budget, rollback de migraciones, caída de push, picos en Sentry y caída de Supabase. Cada runbook sigue la estructura sintoma / diagnostico / contencion / resolucion / postmortem.
4. **Documentación de API** ([dartdoc](#) **pública en GitHub Pages**). Generada automáticamente a partir de los comentarios `///` del código Dart y desplegada en cada release.

7. Conclusiones de evaluación y lecciones aprendidas

El proceso de evaluación arroja cinco lecciones que conviene fijar como aprendizaje del proyecto:

1. **La documentación tratada como contrato verificable evita el drift**. Mientras las cifras se copiaron a mano entre documentos, aparecieron cinco valores distintos del número de tests. Una vez consumidas todas desde [verify_state.sh](#), el drift desaparecio.
2. **Una auditoria independiente con trece sub-agentes en paralelo detecta lo que la auto-revisión nunca encuentra**. La pasada del 22 de mayo localizo deuda crítica (observabilidad inexistente, contraste fallido, factoria duplicada) que la auto-revisión habia normalizado.
3. **La asistencia IA como colaboradora exige gobernanza**. Sin un anchor canonico, sin [grep](#) verificable y sin la regla de "no inventar cifras", la asistencia IA tiende a producir documentos coherentes pero falsos. Con esas tres reglas, su contribucion es trazable y defendible.
4. **Un plan v2 dividido en seis fases con criterios de aceptacion explicitos permite progreso medible**. Cada fase del plan de remediacion tenia criterios de aceptacion expresables como un

`grep` o un test, lo que permitio coordinar trabajos en paralelo sin perder el control.

5. El cierre real (commit + grep) y el cierre documental son distintos; solo el primero cuenta. Esta disciplina ha sido el principal factor diferenciador entre lo declarado y lo entregable.

El proyecto cumple los objetivos académicos definidos en la fase de planificación y deja, al cierre del anchor, un sistema documentado, auditado y verificable, con un plan público para llevarlo desde "TFG aprobado" hasta "producto en producción".

Adenda — Evolución entre 23/05 y 04/06 de 2026

Tras el cierre del anchor original `b908f3c`, se ejecutaron 94 commits adicionales agrupados en cinco oleadas de estabilización post-MVP. Los hechos más relevantes para esta sección de evaluación son los siguientes:

Cierre de un crash bloqueante identificado en pruebas de usuario. Durante las sesiones de prueba con dispositivos físicos del 24-27 de mayo se detectó que la combinación simultánea de varias opciones de accesibilidad (dislexia + alto contraste + escala de texto máxima) provocaba un crash nativo del engine de Flutter no capturable por `try/catch` en Dart. La aplicación quedaba inutilizable y la única recuperación era `adb shell pm clear`. La mitigación implementada (BootCanary + persistencia diferida de preferencias + RecoveryScreen) elimina el riesgo: la app detecta el crash al siguiente arranque, revierte la última preferencia tóxica y, tras dos crashes consecutivos, muestra una pantalla de recovery con `MaterialApp` propio sin shaders ni fuentes custom. El indicador "número de crashes que requieren clear data" pasa de no-cero a cero en los criterios de aceptación operativos.

Resolución de un error PostgreSQL 42P17 detectado en el flujo post-login. La carga de "Mis contribuciones" en el perfil fallaba con `PostgresException: infinite recursion detected in policy for relation route_shares`. Auditadas las políticas RLS de las tres tablas involucradas (`route_shares`, `routes`, `user_routes`), se identificó un ciclo cerrado entre `route_shares_select_owner` y `routes_select_visible`. Se desplegó la migración SQL `fix_route_shares_rls_recursion` que introduce la función `public.is_route_owner(uuid)` con `SECURITY DEFINER` para romper la recursión. Verificación: consulta `SELECT count(*) FROM route_shares` ejecuta sin error tras el deploy. Aumenta en uno el contador de migraciones SQL aplicadas (a 15 totales).

Release pública v1.11.0 con APK como Release Asset en GitHub. Hasta la fecha del anchor original, las versiones del APK se versionaban en el directorio `presentation/public/` del repositorio para servir las desde GitHub Pages. Esta práctica generaba dos problemas: (a) GitHub avisaba (>50 MB) y rechazaría (>100 MB) en versiones futuras; (b) cada APK histórico inflaba el `.git/` indefinidamente. La solución implementada elimina los 9 APKs históricos del `HEAD` (~792 MB liberados), añade `*.apk` a `.gitignore` raíz, y publica el APK v1.11.0 como asset del release oficial. La presentación web enlaza ahora a `releases/latest`, URL estable que apunta siempre a la última versión publicada.

Bypass documentado de verificación de email. Como deuda explícita por la imposibilidad de configurar SMTP propio dentro del cronograma del TFG, se bypassó la verificación de email obligatoria de Supabase. El listener emite `AuthAuthenticated` sin esperar a `emailConfirmedAt`, y `signUpWithEmail` realiza un auto-login defensivo si la configuración del dashboard fuerza la creación

de usuarios sin sesión activa. La infraestructura para reactivar verificación (`EmailVerifyPendingScreen` , ruta `/verify-email`) queda intacta y documentada en `docs/SUPABASE_SETUP.md` .

Logs `warn` y `error` activos en builds de release. Para diagnosticar el bug del flujo Google Sign-In (descrito en `04_desarrollo_implementacion.md` §11.2) fue necesario eliminar el guard `kDebugMode` en los métodos `warn` y `error` de `AppLogger` . Esta decisión permite observabilidad en producción de errores reales sin inflar `logcat` con ruido (los niveles `debug` , `info` y `perf` siguen siendo debug-only). Refuerza el objetivo no funcional de observabilidad.

Los indicadores cuantitativos del anchor original permanecen vigentes y se complementan con: **release v1.11.0 publicada y descargable, migraciones SQL: 14 → 15, +94 commits, +5 documentos técnicos** en `docs/historico/` (planes de acción de las cinco oleadas), **0 crashes bloqueantes que requieran clear data** tras la mitigación, y **3 widgets configurables vía perfil** con preview en vivo.

Adenda 2 — Cierre a 8 de junio de 2026 (release v1.12.1)

Tras la adenda anterior, el trabajo se concentró en cerrar los objetivos funcionales que aún se cubrían parcialmente y en publicar la documentación del proyecto. Hechos relevantes para la evaluación:

Notificaciones push reales (FCM) verificadas con la app cerrada. Se completó la integración de cliente Firebase Cloud Messaging y el registro del token en `device_tokens` ; el backend de envío (`send_notification` + triggers) ya existía. Verificación en dispositivo físico: el log muestra `FlutterFirebaseMessagingBackgroundService started` y la app inicia sin fallos con FCM activo. Queda como dependencia externa la *service account* del operador (`docs/FCM_SETUP.md`), análoga al resto de bloqueadores de `docs/EXTERNAL_BLOCKERS.md` .

Modo conductor resiliente en segundo plano. Migración del seguimiento GPS a `Geolocator.getPositionStream` con foreground service: la posición sigue emitiéndose con la pantalla bloqueada. Indicador operativo "el conductor deja de compartir al bloquear el móvil" pasa de verdadero a falso.

Planificador origen→destino reactivado (`RoutePlannerService` , ruta `/route-plan`), cerrando el flujo de consulta de trayectos.

Publicación de la web del proyecto en GitHub Pages con la presentación, los entregables del TFG navegables y la descarga del APK resuelta desde la release más reciente. Corrección del modo claro de la web.

Evolución de métricas respecto al anchor original: tests **619 → 679**, migraciones SQL **14 → 51**, Edge Functions **4 → 8**, claves ARB **628 → 642**, jobs CI **4 → 6**, release **v1.11.0 → v1.12.1 (APK universal)**. Todas las cifras son verificables sobre el árbol en `master @ b47180d0` .

Nota de autoría: se corrige el nombre del autor a **Koldo Uruburu**, en coherencia con el resto de la documentación y los metadatos del proyecto.

06 — Manual Técnico

Proyecto: Transitly (nexto-stop-v2) **Rama / HEAD original:** `master @ b908f3c` (2026-05-23) **Rama / HEAD actualizado:** `master @ b47180d0` (2026-06-08) **Release distribuable:** v1.12.1 (APK universal) — descargable desde <https://github.com/astralk9999/Transitly/releases/tag/v1.12.1> **Plataformas objetivo:** Android (minSdk 24 / Android 7.0+, targetSdk 34, compileSdk 36), iOS 16.0+, Web (PWA experimental).

Este manual recoge las instrucciones de **instalación, configuración, despliegue y mantenimiento** de Transitly desde la perspectiva de un desarrollador o de un administrador técnico. Para el uso final de la aplicación vease `07_manual_usuario.md`.

1. Visión general de la arquitectura

Transitly es una aplicación Flutter con backend Supabase. Las decisiones estructurales están documentadas como ADRs ([docs/adr/](#)) y son la referencia normativa:

- **ADR 001 — Riverpod 2.6** como gestor de estado, con uso explícito de `autoDispose` y `family` en las features que lo requieren.
- **ADR 002 — Freezed** para todos los modelos del dominio (veintisiete modelos inmutables con `copyWith`, `==`, `hashCode` y soporte `json_serializable`).
- **ADR 003 — Hive 2.2** con `HiveAesCipher` y tres boxes principales como cache local cifrada.
- **ADR 004 — Supabase** como backend (PostgreSQL, Auth, Realtime, Storage y Edge Functions Deno).
- **ADR 005 — Feature-first** como organizacion del código en `lib/features/`.

A fecha de defensa el proyecto suma **veintisiete features** (446 ficheros `.dart`, ~94k LOC), **cincuenta y una migraciones SQL consecutivas**, **ocho Edge Functions** desplegadas, **seis runbooks operativos**, **679 tests** en verde, **642 claves ARB** localizadas a ES/EN/AR y **seis jobs CI** (analyze, test, build web, build APK, semgrep, gitleaks). El cuadro de mando interno marca **TFG 8,9 / 10** y **Produccion 6,0 / 10**, diferenciando con claridad la madurez académica de la madurez productiva.

2. Requisitos del sistema de desarrollo

2.1. Software base

Herramienta	Versión recomendada	Notas
Flutter SDK	3.16+ (alíneado con la versión pinneada en CI)	Trae Dart 3 incluido.
Dart SDK	3.2+	Si se usa Flutter ≥ 3.16 , viene integrado.
Android Studio	Hedgehog o superior con Android SDK API 34	Incluye <code>platform-tools</code> , <code>cmdline-tools</code> , NDK opcional.
Xcode	15+	Solo macOS, requerido para compilar a iOS.
Java / JDK	17 LTS	Necesario para Gradle moderno.
Git	2.40+	El proyecto asume <code>git switch</code> , <code>git restore</code> .
Node.js	20 LTS	Para tooling auxiliar (Gitleaks local, scripts JS).
Deno	1.40+	Necesario para ejecutar las Edge Functions en local.
Supabase CLI	1.x	Para <code>supabase link</code> , <code>db push</code> , <code>functions deploy</code> .
Cuenta Supabase	Plan gratuito o superior	Proyecto con PostgreSQL, Auth, Realtime y Storage activos.
Cuenta Firebase	Plan Blaze (FCM HTTP v1) o gratuito según uso	Necesario para FCM.

2.2. Sistema operativo

Soportado en Windows 10/11, macOS 12 o superior y Linux Ubuntu 20.04 o superior. La compilación a iOS exige macOS por requisito de la cadena de Apple.

3. Instalación paso a paso

3.1. Clonado del repositorio

```
git clone https://github.com/astralk9999/Transitly.git
cd Transitly
```

3.2. Variables de entorno

A partir del refuerzo de seguridad SEC2, las claves se inyectan en tiempo de compilación mediante `--dart-define` (el fichero `.env` ya no se distribuye como asset). El repositorio incluye `.env.example` como plantilla de referencia con explicación de cada variable.

```
cp .env.example .env    # solo como referencia local; no se bundlea
```

Variables obligatorias para arranque:

- `SUPABASE_URL`
- `SUPABASE_ANON_KEY`

Variables opcionales (degradación silenciosa si faltan):

- `SUPABASE_FUNCTIONS_URL` (por defecto `<url>/functions/v1`).
- `POSTHOG_API_KEY`, `POSTHOG_HOST`.
- `SENTRY_DSN`.
- `MAPTILER_API_KEY` (necesaria para teselas en línea fuera de la región offline).

3.3. Dependencias y generación de código

```
flutter pub get
flutter pub run build_runner build --delete-conflicting-outputs
flutter gen-l10n
```

`flutter pub get` resuelve aproximadamente 150 paquetes (Riverpod 2.6, supabase_flutter, hive 2.2, flutter_map, sentry 8, posthog 5, flutter_secure_storage, very_good_analysis, leak_tracker_flutter_testing, entre otros). `build_runner` regenera los Freezed y `json_serializable`. `flutter gen-l10n` produce las clases tipadas de localización a partir de los ARB.

3.4. Firebase Cloud Messaging

El proyecto Firebase es `transitly-ee8cf`. El repositorio ya incluye `lib/firebase_options.dart` con las claves del proyecto; el fichero `android/app/google-services.json` **no se versiona** (está en `.gitignore`) y debe colocarse manualmente o regenerarse con:

```
flutterfire configure --project=transitly-ee8cf --platforms=android
```

Esto produce/actualiza `lib/firebase_options.dart` y `android/app/google-services.json`. Sin el `google-services.json` el `firebase_setup.dart` degrada silenciosamente: la app funciona sin push. Para el **envío programático** desde el backend hace falta además la *service account* de Firebase como secreto `FCM_SERVICE_ACCOUNT_JSON` (ver `docs/FCM_SETUP.md`).

3.5. Ejecución en debug

```
flutter run \
  --dart-define=SUPABASE_URL=https://<tu-proyecto>.supabase.co \
  --dart-define=SUPABASE_ANON_KEY=eyJh...
```

3.6. Build release

```
# APK universal (el publicado en v1.12.1; máxima compatibilidad de dispositivos)
flutter build apk --release --dart-define-from-file=dart_defines.json

# Alternativa por ABI (binarios más pequeños) o App Bundle para Play Store
flutter build apk --release --split-per-abi \
  --obfuscate --split-debug-info=build/debug-info \
  --dart-define-from-file=dart_defines.json
```

El binario se firma con la keystore real si existe `android/key.properties`; en caso contrario, se firma con la debug key y el artefacto no es p ublicable. El APK universal de v1.12.1 ocupa ~91 MB (incluye arm64-v8a, armeabi-v7a y x86_64).

4. Configuraci n avanzada

4.1. Feature flags y constantes de compilacion

Los `--dart-define` se utilizan tambi n como **feature flags**. Las flags actuales incluyen `ENABLE_NFC`, `ENABLE_WIDGETS_NATIVE`, `ENABLE_REALTIME_DRIVER` y `OPERATOR_DEFAULT_SLUG`. Se ajustan en la l nea de compilacion sin tocar c digo.

4.2. Cambiar el operador por defecto

El operador por defecto es COMUJESA (`comujesa`). Para cambiarlo:

1. Insertar el operador objetivo en la base mediante `dart tools/seed_operators.dart`.
2. Pasar `--dart-define=OPERATOR_DEFAULT_SLUG=<slug>` en el build.

4.3. Cambiar la paleta de marca

Las paletas viven en `lib/core/theme/palettes/`. Cada paleta declara los tokens `surfaceHi/Mid/Lo`, `textHi/Mid/Lo`, `primary`, `accent` y `signal*`. Tras editar o anadir una nueva paleta:

1. Ejecutar `dart run tool/contrast_check.dart` para validar WCAG AA.
2. Anadir el caso al test `accessibility_matrix_test.dart`.
3. Regenerar `00_MAESTRO.md` con `tool/verify_state.sh`.

4.4. Anadir un idioma

1. Crear `lib/l10n/app_<c digo>.arb` con todas las claves traducidas (628 en el anchor).
2. Anadir el c digo en `l10n.yaml`.
3. Si es RTL, anadir el c digo en `RTL_LOCALES` (`lib/core/utils/locale_utils.dart`).
4. Ejecutar `flutter gen-l10n`.

5. Estructura del repositorio

nexto-stop-v2/	
├─ lib/	Código Dart (446 ficheros, ~94k LOC)
│ └─ main.dart	Bootstrap (Env → Hive → Supabase → Firebase → ProviderScope)
│ └─ app.dart	MaterialApp.router + theme + locale
│ └─ core/	router, theme, utils, observability
│ └─ data/	12 entidades con repositorio canonico, cache Hive,
│ │	realtime channel manager, push, sync, NFC
│ └─ features/	27 features con `*_screen.dart`
│ └─ l10n/	ARB ES/EN/AR + generated (642 claves)
│ └─ shared/	38 widgets reutilizables, 30 models Freezed, providers
├─ test/	679 tests
├─ supabase/	
│ └─ migrations/	51 migraciones consecutivas
│ └─ functions/	8 Edge Functions Deno
├─ android/	Kotlin DSL, signing condicional, foreground service
├─ ios/	Info.plist, entitlements
├─ web/	PWA experimental
├─ astro/	Web de producto (landing + app Flutter en /app, SSR)
├─ presentation/	Web del TFG (GitHub Pages: presentación + entregables)
├─ docs/	
│ └─ 00_MAESTRO.md	Fuente única de verdad
│ └─ adr/	5 ADRs vivos
│ └─ runbooks/	6 runbooks operativos
│ └─ historico/	Auditorias y planes archivados
│ └─ tfg/	Memoria académica (01..08)
├─ tool/ y tools/	Scripts Dart, shell y Node (verify_state, contrast, seed,
apply_sql)	
└─ .github/workflows/	CI (6 jobs) + deploy de Pages + dartdoc + release-please

6. Backend Supabase

6.1. Vinculacion con el proyecto

```
supabase login
supabase link --project-ref <ref>
```

6.2. Aplicación de migraciones

```
supabase db push
supabase migration list
```

Las **cincuenta y una migraciones consecutivas** (`001_init.sql` en adelante) cubren: schema base, RLS default-deny, parches de `search_path`, storage, RPCs, helpers de votos, helpers de invitación, triggers de notificaciones, tokens FCM, triggers push, auditoria extendida, reputación, exportacion offline, exportaciones GDPR, sistema de reputación/XP, avisos geo, líneas y horarios de COMUJESA,

conductor en vivo (`driver_live_trips`), zonas y la función `is_route_owner` que rompe la recursión RLS 42P17.

6.3. Despliegue de Edge Functions

Las **ocho funciones desplegadas** son `send_notification`, `import_gtfs`, `delete_user`, `purge_old_data`, `generate_data_export`, `approve_user_route`, `promote_stop_to_official` y `validate_share_code`. Las invocadas por el usuario verifican JWT; las invocadas por cron o triggers internos (autenticados por `service_role`) se despliegan sin verificación:

```
supabase functions deploy send_notification          # invocada por triggers (service_role)
supabase functions deploy delete_user --verify-jwt
supabase functions deploy generate_data_export --verify-jwt
supabase functions deploy validate_share_code --verify-jwt
supabase functions deploy approve_user_route --verify-jwt
supabase functions deploy promote_stop_to_official --verify-jwt
supabase functions deploy purge_old_data --verify-jwt
supabase functions deploy import_gtfs --no-verify-jwt
```

6.4. Secretos en Supabase

```
supabase secrets set FCM_SERVICE_ACCOUNT_JSON="$(cat fcm-service-account.json)"
supabase secrets set SENTRY_DSN="https://..."
supabase secrets set ALLOWED_ORIGINS="https://transitly.app,https://app.transitly.app"
```

6.5. Cron interno

Para `purge_old_data` y `delete_user_worker` se habilita `pg_cron` (requiere Supabase Pro) y se programan ejecuciones diarias con la zona horaria del proyecto.

7. CI/CD

El workflow `.github/workflows/ci.yml` define **seis jobs principales** ejecutados en cada push y en cada pull request hacia `master`:

1. **Flutter Analyze** — bloqueo total ante cualquier error, warning o info.
2. **Flutter Test** — `flutter test --coverage`, validacion de umbral global y upload a Codecov.
3. **Build Web (release)** — compilacion del target web.
4. **Build Android APK / AAB** — APK con `--split-per-abi`, `--obfuscate` y `--split-debug-info`, validacion del budget de tamaño, archivado de symbol files y firma del AAB.

A estos se anaden dos jobs auxiliares: **Gitleaks** (escaneo de secretos en el repositorio) y **Semgrep** (SAST con reglas locales en `.semgrep/rules.yaml`).

7.1. Secrets necesarios en GitHub

- `ANDROID_KEYSTORE_BASE64`

- `ANDROID_KEY_PROPERTIES_BASE64`
- `GOOGLE_SERVICES_JSON_BASE64`
- `GOOGLE_SERVICES_INFO_PLIST_BASE64`
- `SUPABASE_ACCESS_TOKEN`
- `SUPABASE_URL` , `SUPABASE_ANON_KEY`
- `SENTRY_AUTH_TOKEN`
- `POSTHOG_API_KEY`
- `CODECOV_TOKEN`

7.2. Releases con `release-please`

El flujo de release es: anotacion de cambios con Conventional Commits, tag `vX.Y.Z` , `release-please` abre PR de release, al fusionarse se publica GitHub Release con el `CHANGELOG.md` actualizado y se adjuntan los artefactos (APK por ABI, AAB firmado, debug-info).

8. Mantenimiento rutinario

8.1. Actualizacion mensual de dependencias

```
flutter pub outdated
flutter pub upgrade --major-versions
flutter pub run build_runner build --delete-conflicting-outputs
flutter analyze && flutter test
```

Las actualizaciones mayores (Riverpod 3, Sentry 9, etc.) se ejecutan en rama separada con migración documentada en `docs/MEGA_PLAN_REFINAMIENTO.md` .

8.2. Rotacion de claves

La `anon key` y los PAT de Supabase se rotan cada **seis meses**, siguiendo el procedimiento de `docs/SECURITY_PAT_ROTATION.md` . La rotacion implica generar la nueva clave, actualizar secrets en GitHub, regenerar el build de release y revocar la clave anterior.

8.3. Auditorias trimestrales

Cada trimestre se ejecutan tres scripts de verificación:

- `tool/verify_state.sh` — confirma cifras del anchor.
- `tool/check_no_hardcoded.sh` — busca strings hardcodeados fuera de l10n.
- `dart run tool/contrast_check.dart` — válida contraste WCAG AA de todas las paletas.

8.4. Backup de la keystore Android

La keystore de release es irreemplazable: una perdida obliga a publicar la app como una nueva entrada en Play Store. Por ello se guarda en **tres ubicaciones independientes** (gestor de secretos

personal, copia cifrada offline y backup en almacenamiento frio), y nunca se elimina del repositorio personal de claves.

9. Runbooks operativos

Los seis runbooks de `docs/runbooks/` describen incidentes habituales en producción. Resumen por archivo:

- `disaster_recovery.md` . Procedimiento de recuperacion ante perdida de datos o de proyecto Supabase: restaurar desde el último `pg_dump` validado, comprobar consistencia de RLS y notificar a usuarios via push y banner in-app.
- `error_budget_policy.md` . Define el error budget mensual (objetivo 99,5 % de exito en arranques en frio y 99,0 % en envio de push) y la política de freeze de releases cuando se consume el 75 % del budget.
- `migration_rollback.md` . Pasos para revertir una migración erronea: marcar como `down` la migración en `supabase/migrations/_meta` , aplicar el inverso manual y reanunciar el estado al equipo. Incluye el aprendizaje de la incidencia 04/05/2026.
- `push_down.md` . Diagnostico cuando FCM deja de enviar: validar el JWT firmado, comprobar la cuota del proyecto Firebase, verificar el ratio de errores en `device_tokens` y forzar reenvio del token desde la app.
- `sentry_spike.md` . Procedimiento ante un pico de errores en Sentry: bisecar la versión implicada con `release-please` , aplicar un kill-switch via feature flag remoto si procede y abrir incidencia con plantilla postmortem.
- `supabase_down.md` . Actuacion ante caida del backend: activar banner offline en la app (ya cableado), confirmar el estado en `status.supabase.com` y, si la caida supera quince minutos, comunicar a usuarios y replanificar la sesión de demo.

10. Resolucion de problemas frecuentes

Sintoma	Causa probable	Solucion
Firebase <code>initializeApp</code> falla en arranque.	Falta <code>firebase_options.dart</code> o <code>google-services.json</code> .	Ejecutar <code>flutterfire configure --project=transitly-prod</code> . Si no se desea push, dejar el degradado silencioso del <code>firebase_setup.dart</code> .
RLS responde con error <code>42501</code> .	Política de Row Level Security denegada por falta de rol.	Revisar <code>is_admin()</code> , <code>is_moderator_or_admin()</code> y el campo <code>role</code> en <code>profiles</code> .
<code>flutter build apk --release</code> falla con duplicado <code>androidx.work</code> .	Reintroduccion accidental de <code>workmanager</code> .	Eliminar el plugin del <code>pubspec.yaml</code> . Esta es la incidencia historica de 22/04/2026.

Sintoma	Causa probable	Solucion
Los tests de <code>integration_test</code> fallan en CI.	Emulador inadecuado o falta <code>integration_test</code> en <code>dev_dependencies</code> .	Usar emulador Pixel 6 API 34 y confirmar la dependencia.
Las teselas del mapa no aparecen sin red.	<code>FMTCSoreDirectory</code> no inicializado en el arranque.	Inicializar <code>FMTCSore</code> antes de <code>runApp</code> en <code>main.dart</code> .
Hive lanza <code>HiveError: Cipher mismatch</code> .	Se abrio un box antes de inicializar <code>flutter_secure_storage</code> .	Asegurar el orden: <code>SecureStorage</code> → <code>HiveAesCipher</code> → <code>Hive.openBox</code> .
Sentry no captura nada en release.	Uso de <code>print</code> en lugar de <code>AppLogger.error</code> .	Sustituir <code>print</code> por <code>AppLogger.error(...)</code> ; en release los breadcrumbs se anaden fuera del bloque <code>kDebugMode</code> .

11. Versiónado y release notes

El proyecto sigue **SemVer** (`MAJOR.MINOR.PATCH`) y aprovecha **Conventional Commits** como entrada para `release-please`. El flujo es:

1. Todos los commits hacia `master` usan los tipos `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`, `perf:` o `build:`.
2. `release-please` mantiene una pull request de release abierta con la versión siguiente y el `CHANGELOG.md` autogenerado.
3. Al fusionar la PR, se crea el tag `vX.Y.Z` y la `GitHub Release` con los artefactos adjuntos.
4. El `CHANGELOG.md` queda commiteado y trazable.

12. Referencias internas

- `docs/00_MAESTRO.md` — fuente única de verdad de las cifras del proyecto.
- `docs/adr/` — cinco ADRs vivos.
- `docs/runbooks/` — seis runbooks operativos.
- `docs/MEGA_PLAN_REFINAMIENTO.md` — plan vivo con clasificacion P0-P3.
- `docs/EXTERNAL_BLOCKERS.md` — diecinueve bloqueadores externos al alcance individual.
- `docs/SECURITY_PAT_ROTATION.md` — rotacion de PAT y claves anon.
- `docs/RELEASE_CHECKLIST.md` — comprobaciones previas al release.
- `docs/historico/AUDIT_2026_05_22.md` — auditoria deep-dive con trece sub-agentes.
- `docs/historico/PLAN_ACCION_REMEDIACION_v2.md` — plan v2 en seis fases.
- `AGENTS.md` — guía operativa para sesiones de desarrollo asistido por IA.
- `android/README.md` — flujo de firma de release y Play App Signing.

Adenda — Procedimientos añadidos entre 23/05 y 04/06 de 2026

Distribución del APK como GitHub Release Asset

Desde la versión v1.11.0 los APKs no se versionan en el repositorio sino que se publican como assets de release en GitHub. El flujo es:

1. Generar el APK release: `flutter build apk --release --dart-define-from-file=dart_defines.json`.
2. Crear el tag local: `git tag vX.Y.Z <commit>` (por convención el commit del cierre de la versión).
3. Push del tag: `git push origin vX.Y.Z`.
4. Crear el release con el APK:

```
$apk = "C:\Users\<user>\AppData\Local\Temp\transitly-vX.Y.Z.apk"
& "C:\Program Files\GitHub CLI\gh.exe" release create vX.Y.Z `
  "$apk#Transitly vX.Y.Z (Android APK)" `
  --title "vX.Y.Z - resumen breve" `
  --notes "<notas en markdown>"
```

5. Verificar que <https://github.com/astralk9999/Transitly/releases/latest> redirige al nuevo release.

La presentación web ([presentation/src/](#)) enlaza siempre a [releases/latest](#) , por lo que no requiere rebuild tras un release nuevo.

Aplicación de migraciones SQL adicionales

Las migraciones SQL posteriores al anchor original se aplican vía MCP de Supabase o directamente desde el SQL Editor del dashboard. Para auditar políticas RLS existentes ante un sospechoso de recursión 42P17:

```
SELECT schemaname, tablename, policyname, cmd, qual, with_check
FROM pg_policies
WHERE tablename IN ('<tabla_1>', '<tabla_2>', '<tabla_3>')
ORDER BY tablename, policyname;
```

La migración [fix_route_shares_rls_recursion](#) (2026-06-04) introduce la función [public.is_route_owner\(uuid\)](#) con `SECURITY DEFINER STABLE` para romper ciclos en políticas de visibilidad. Documentación operativa completa en [docs/SUPABASE_SETUP.md](#) sección "RLS Policies — gotchas conocidos".

Recuperación ante crash de arranque

Si la app no arranca tras un cambio en preferencias de accesibilidad:

1. **Recuperación automática esperada:** al detectar dos crashes consecutivos en arranque, [BootCanary](#) activa [RecoveryScreen](#) con tres acciones (restaurar valores por defecto, continuar sin cambios, reportar el problema).
2. **Recuperación manual de último recurso:** `adb shell pm clear com.transitly.transitly` borra todas las preferencias y caches y permite arrancar con la configuración inicial.

Bypass temporal de verificación de email

Mientras no se configure SMTP propio:

- **Supabase Dashboard** → **Authentication** → **Providers** → **Email**: toggle "Confirm email" en OFF.
- **Cuentas históricas sin verificar** (creadas antes del bypass): ejecutar en SQL Editor `UPDATE auth.users SET email_confirmed_at = now() WHERE email_confirmed_at IS NULL;` para desbloquearlas.
- **Reactivar verificación cuando se disponga de SMTP**: seguir los pasos descritos en `docs/SUPABASE_SETUP.md` sección "Estado actual (2026-06-04)" → "Reactivar verificación cuando se configure SMTP".

Diagnóstico en builds release

Desde el 04/06 los niveles `warn` y `error` de `AppLogger` emiten a `logcat` también en builds release (los niveles `debug`, `info` y `perf` siguen siendo debug-only). Para capturar logs filtrados por el proceso de la app:

```
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
$pid_app = & $adb -s <device-id> shell pidof com.transitly.transitly
& $adb -s <device-id> logcat -d --pid=$pid_app | Select-String "Auth|WARN|ERROR"
```

Adenda 2 — Procedimientos de la release v1.12.1 (8 de junio de 2026)

Activación del push FCM en el cliente

1. Colocar `android/app/google-services.json` del proyecto `transitly-ee8cf` (no se versiona).
2. Verificar que el plugin está activo en `android/app/build.gradle.kts`: `id("com.google.gms.google-services")` (el classpath ya está en `android/build.gradle.kts`).
3. `lib/firebase_options.dart` ya contiene las claves reales del proyecto.
4. Compilar e instalar. La app registra el token en `device_tokens` al iniciar sesión; verificación en `logcat`: `FlutterFirebaseMessagingBackgroundService started`.
5. Para el envío programático, configurar la *service account* como secreto de la Edge Function (ver `docs/FCM_SETUP.md`):

```
supabase secrets set FCM_PROJECT_ID=transitly-ee8cf
supabase secrets set FCM_SERVICE_ACCOUNT_JSON="$(cat service-account.json)"
```

Modo conductor en segundo plano (foreground service)

El seguimiento usa `Geolocator.getPositionStream` con

`AndroidSettings.foregroundNotificationConfig`. Requiere en `AndroidManifest.xml` los permisos `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION` y `WAKE_LOCK` (ya añadidos). No requiere paquetes adicionales.

Publicación de la release y la web (GitHub Pages)

1. Compilar el APK universal: `flutter build apk --release --dart-define-from-file=dart_defines.json`.
2. Crear la release y subir el APK como asset (vía `gh release create` o la API REST de GitHub) con un nombre claro (`transitly-vX.Y.Z.apk`); marcarla como *latest*.
3. La web (`presentation/`) se despliega sola al hacer push a `master` tocando `presentation/**` (workflow `deploy-presentation.yml` → rama `gh-pages`). Los botones `[data-download-apk]` resuelven el APK desde `releases/latest` vía la API de GitHub, por lo que la web no necesita reconstruirse tras cada release.

07 — Manual de Usuario

Aplicación: Transitly **Versión del documento original:** 2026-05-23 **Versión actualizada:** 2026-06-08
Versión instalable distribuida: v1.12.1 (APK universal, Android 7.0+) — descarga directa desde <https://github.com/astralk9999/Transitly/releases/latest> o desde la web del proyecto <https://astralk9999.github.io/Transitly/> **Plataformas:** Android, iOS y Web (PWA) **Audiencia:** usuario final, sin conocimientos técnicos previos

1. Que es Transitly

Transitly es una aplicación móvil y web que te ayuda a usar el transporte público urbano e interurbano en ciudades medias españolas. Te permite consultar las líneas y paradas de tu ciudad, ver donde están los autobuses en el mapa, conocer los próximos tiempos de llegada, reportar incidencias y sugerir mejoras al servicio. La aplicación se ha desarrollado tomando como operador piloto a COMUJESA (Jerez de la Frontera), aunque su diseño permite incorporar nuevos operadores en el futuro.

Transitly busca cubrir el hueco que existe en ciudades sin aplicaciones oficiales en tiempo real. Por eso combina datos de horario con la información que aportan los propios conductores y los pasajeros, ofrece modo sin conexión para zonas con cobertura débil y se ha diseñado para ser accesible: alto contraste, lectores de pantalla, escala de texto, modo daltonismo y soporte de español, inglés y árabe con lectura de derecha a izquierda.

2. Instalación

2.1. Android (instalación del APK)

Durante la fase de TFG, Transitly se distribuye como APK (aún no está publicada en Google Play). La instalación es sencilla:

1. Desde tu teléfono Android, entra en la web del proyecto <https://astralk9999.github.io/Transitly/> y pulsa **Descargar APK** (o ve directo a <https://github.com/astralk9999/Transitly/releases/latest>).
2. Cuando termine la descarga, abre el archivo `transitly-vX.Y.Z.apk` .
3. Android te avisará de que procede de un origen desconocido: pulsa **Ajustes** y concede permiso para instalar (puedes revocarlo después).
4. Pulsa **Instalar** y, al terminar, **Abrir**. El icono de Transitly quedará en tu escritorio.

Requiere **Android 7.0 o superior**. El APK es universal, así que funciona en cualquier móvil

(arm64, arm de 32 bits o x86_64).

2.2. iOS

1. Abre la App Store en tu iPhone o iPad.
2. Busca **Transitly** o entra en <https://transitly.app/ios>.
3. Pulsa **Obtener** y confirma con Face ID, Touch ID o contraseña.
4. Cuando termine la instalación, pulsa **Abrir**.

2.3. Web (PWA)

1. Entra desde tu navegador en <https://transitly.app/web>.
2. En el menú del navegador, selecciona **Instalar aplicación** o **Añadir a pantalla de inicio**.
3. Confirma cuando el navegador te lo pida. Tendrás un icono igual que el de una aplicación nativa.

Si tu teléfono no es compatible con la instalación nativa, la versión web funciona igualmente desde cualquier navegador moderno.

3. Primera vez que abres la aplicación

La primera vez que ejecutes Transitly veras tres pantallas de bienvenida que explican brevemente las funciones principales. Puedes deslizar el dedo para avanzar o pulsar **Saltar** si prefieres ir directo a la aplicación.

A continuación se te pedirán los siguientes pasos:

1. **Selección ciudad.** Por defecto aparece **Jerez de la Frontera**. Si quieres cambiarla, pulsa sobre el nombre y elige otra de la lista.
2. **Concede permisos.** La aplicación solicita ubicación (para mostrar tu posición en el mapa), notificaciones (para avisos de retrasos) y, solo en Android, acceso a NFC (para leer la tarjeta del Consorcio). Puedes denegarlos y la aplicación seguirá funcionando con menos funciones.
3. **Banner de consentimiento (privacidad).** Veras una pantalla resumiendo los tratamientos opcionales: analítica de uso, informes de fallos y comunicaciones. Cada uno tiene un interruptor independiente, todos desactivados por defecto. Acepta o rechaza con los botones inferiores.
4. **Iniciar sesión o continuar como invitado.** Pulsa **Continuar como invitado** para usar la aplicación sin registro, o **Iniciar sesión** para acceder a funciones avanzadas (favoritos sincronizados, reportar incidencias verificadas, modo conductor con código, etc.).

4. Pantalla principal

La pantalla de inicio tiene cinco pestañas en la barra inferior:

- **Inicio.** Resumen personalizado: tus líneas favoritas, últimas alertas y accesos rápidos a planificar un viaje.
- **Mapa.** Mapa interactivo con paradas y autobuses en circulación.
- **Tarjeta.** Lectura del saldo de la tarjeta del Consorcio Andalucía mediante NFC.
- **Buscar.** Buscador de líneas, paradas y direcciones.
- **Perfil.** Tu cuenta, ajustes, privacidad, idioma y accesibilidad.

Para cambiar de pestaña, pulsa el icono correspondiente. La pestaña activa aparece resaltada.

5. Consultar una línea

1. Ve a **Buscar** o pulsa una línea desde **Inicio**.
 2. Aparecerá la pantalla de **Detalle de línea** con:
 - El recorrido en el mapa.
 - La lista de paradas de ida y vuelta.
 - Los horarios programados.
 - Las alertas activas (desvíos, cortes, refuerzos).
 3. Si quieres guardar la línea, pulsa el icono de **estrella** en la esquina superior derecha. Aparecerá entre tus favoritos en Inicio.
-

6. Ver un autobús en el mapa

1. Pulsa la pestaña **Mapa**.
 2. Cada parada aparece como un punto y cada autobús en circulación como un icono de bus.
 3. Pulsa un autobús para ver su línea, sentido y próxima parada.
 4. Pulsa una parada para ver las próximas llegadas en esa parada.
 5. Usa los gestos habituales: arrastra para desplazar, junta o separa los dedos para ampliar o reducir.
-

7. Proximas llegadas

Cuando entras en una parada, ya sea desde el mapa o desde el detalle de una línea, verás una lista con las **próximas tres salidas estimadas** de las líneas que pasan por ella. Cada estimación indica:

- La línea y el destino.
- Los minutos restantes.
- Si la estimación procede de horario o de GPS de un conductor.

Si la información es antigua o no está disponible, la aplicación lo indica claramente y no inventa datos.

8. Reportar una incidencia

1. Entra en la parada o en la línea afectada.
2. Pulsa el boton **Reportar**.
3. Elige el tipo de incidencia: **Retraso**, **Vehículo lleno**, **Desvio** o **Avería**.
4. Anade un comentario opcional (máximo 200 caracteres). No incluyas datos personales de terceros.
5. Pulsa **Enviar**. Tu reporte se mostrara en el mapa para el resto de usuarios.

Los reportes se moderan automáticamente y se contrastan con otros usuarios. Si alguien reporta lo mismo, gana reputación en la comunidad.

9. Dejar feedback sobre una ruta

1. Entra en el detalle de la línea.
2. Pulsa **Dar feedback**.
3. Puntua de 1 a 5 estrellas:
 - **Puntualidad** del servicio.
 - **Amabilidad** del personal.
 - **Limpieza** del vehiculo.
4. Anade un comentario opcional.
5. Pulsa **Enviar**.

El operador recibe estos comentarios de forma agregada y anonima.

10. Sugerir una nueva ruta

1. Ve a **Perfil** y pulsa **Sugerir ruta nueva**.
2. Rellena el formulario:
 - **Origen y destino** (puedes elegir paradas existentes o escribir direcciones).
 - **Operador** propuesto.
 - **Paradas intermedias** que crees necesarias.
 - **Justificacion** breve.
3. Pulsa **Enviar sugerencia**.

Otros usuarios podrán votar la sugerencia. Las propuestas con más votos se trasladan al operador.

11. Leer la tarjeta del Consorcio (NFC)

Esta función solo está disponible en Android con NFC y con la tarjeta física del **Consorcio de Transporte de Andalucía**.

1. Pulsa la pestaña **Tarjeta**.
2. Pulsa **Escanear tarjeta**.
3. Acerca la tarjeta a la parte trasera del teléfono y mantenla quieta durante unos segundos.
4. Verás el **saldo actual** y los **últimos viajes** registrados.

La aplicación no recarga la tarjeta ni modifica su contenido: solo lee la información pública grabada en el chip.

12. Usar la aplicación sin conexión

1. Ve a **Perfil > Datos sin conexión**.
2. Pulsa **Descargar región de Jerez**.
3. Espera a que termine la descarga (puede tardar uno o dos minutos en conexión lenta).
4. A partir de ese momento podrás consultar el mapa, las paradas y los horarios aunque no tengas internet. Las llegadas en tiempo real requieren conexión.

Para liberar espacio, en el mismo apartado puedes **Borrar la región descargada**.

13. Cambiar idioma, tema y accesibilidad

Desde **Perfil** tienes tres apartados:

- **Apariencia**. Permite elegir tema claro, oscuro o automático según el sistema, escoger la paleta de colores y la fuente.
- **Accesibilidad**. Activa alto contraste, modo daltonismo (varias matrices disponibles), aumenta la escala de texto y habilita la fuente para dislexia.
- **Idioma**. Seleccióna español, inglés o árabe. Al elegir árabe la aplicación cambia automáticamente a lectura de derecha a izquierda.

Los cambios se aplican al instante y se recuerdan para la próxima vez.

14. Privacidad y protección de datos

Transitly cumple el Reglamento General de Protección de Datos. Tienes control total sobre tu información desde **Perfil > Privacidad**.

14.1. Modificar el consentimiento

En cualquier momento puedes activar o desactivar:

- **Analítica de uso.**
- **Informes de fallos.**
- **Comunicaciones comerciales.**

Todos están desactivados por defecto y nunca afectan al funcionamiento básico de la aplicación.

14.2. Solicitar una copia de tus datos (Artículo 20)

1. Entra en **Privacidad** y pulsa **Solicitar mis datos**.
2. Confirma tu correo electrónico.
3. Recibirás un archivo ZIP con tu información en un plazo máximo de **siete días**.

14.3. Eliminar tu cuenta (Artículo 17)

1. En **Privacidad**, pulsa **Eliminar cuenta**.
2. Confirma tu intención. La cuenta entra en periodo de gracia de **30 días**.
3. Si cambias de opinión, puedes cancelarla volviendo a entrar antes del **día 29**. A partir del día 30 el borrado es irreversible.

14.4. Verificación de edad (Artículo 8)

Al registrarte se solicita la fecha de nacimiento. Los **menores de 16 años** no pueden crear cuenta. Si la introduces y el sistema detecta que no cumples la edad mínima, el registro se cancela.

15. Modo conductor

Si trabajas como conductor o conductora y tu operador te ha facilitado un código de invitación:

1. Ve a **Perfil > Activar modo conductor**.
2. Introduce el código proporcionado por el operador.
3. Al activarse, en **Perfil** aparece un acceso **Panel de conductor** con tu cuadro de turnos.
4. Pulsa **Iniciar ruta** cuando comiences un servicio. El teléfono compartirá tu ubicación solo durante el trayecto.
5. Cuando termines, pulsa **Finalizar ruta**. El historico queda en **Mis viajes** para tu propio control.

Si no eres conductor o conductora, ignora este apartado: la opción no hace nada sin un código válido.

16. Notificaciones

1. Ve a **Perfil > Notificaciones**.
2. Activa o desactiva los avisos por categoría:

- **Alertas de tus líneas favoritas.**
- **Retrasos significativos.**
- **Novedades del servicio.**

3. Si una notificación concreta te molesta, mantenla pulsada para abrir los ajustes del sistema y silenciarla.

17. Reportar un problema con la aplicación

Si encuentras un fallo o una función no se comporta como esperas:

1. Ve a **Perfil > Ayuda > Reportar un problema.**
2. Describe lo ocurrido y, si quieres, adjunta una captura.
3. Pulsa **Enviar.**

También puedes escribir a sophite@transitly.app indicando versión del teléfono y de la aplicación.

18. Atajos de accesibilidad

Transitly esta probada con los lectores de pantalla más habituales:

- **TalkBack (Android).** Activa el lector desde **Ajustes del sistema**

Accesibilidad^{**}. Después, abre Transitly y desplaza el dedo sobre la pantalla para escuchar cada elemento. Doble toque activa la opción enfocada.

- **VoiceOver (iOS).** Actívalo desde **Ajustes > Accesibilidad > VoiceOver**. En Transitly funciona igual: deslizar para enfocar, doble toque para activar.

La aplicación incluye descripciones para todos los iconos importantes, respeta la escala de texto del sistema y mantiene el contraste mínimo exigido en todos los temas, también con paletas alternativas para daltonismo.

19. Preguntas frecuentes

No me aparece mi ciudad en la lista. En la versión actual solo esta disponible Jerez de la Frontera con datos reales. Si necesitas otra ciudad, escribe a sophite@transitly.app.

La aplicación no detecta mi tarjeta NFC. Verifica que el NFC del teléfono este activo, que tu modelo lo soporte y que la tarjeta sea del Consorcio de Transporte de Andalucía. Algunas fundas gruesas dificultan la lectura.

Las llegadas estimadas no son exactas. Cuando un autobús no tiene GPS activo, la aplicación usa el horario teórico, que puede no reflejar retrasos. Reportar incidencias ayuda a mejorar la estimación.

He borrado la aplicación y quiero recuperar mis favoritos. Si te registraste con cuenta, vuelve a iniciar sesión y se restauraran. Si usaste modo invitado, los favoritos se guardan solo en el dispositivo.

Quiero borrar todos mis datos. Sigue el paso 14.3 de este manual.

Adenda — Novedades de la versión v1.11.0 (4 de junio de 2026)

Si vienes de una versión anterior, en v1.11.0 encontrarás los siguientes cambios visibles:

Login con Google funcional. El botón "Continuar con Google" en la pantalla de inicio de sesión ya funciona en todos los dispositivos. Si tu cuenta no entra al primer intento, espera unos segundos y vuelve a tocarla — la app reintentará automáticamente.

Sin obligación de verificar email para usar la app. El flujo de registro envía las credenciales al servidor y entras directamente al inicio sin esperar a un correo de verificación. Esto es temporal mientras se configura el servidor de correo propio; si más adelante recibes un email de verificación, púlsalo igualmente para mantener la cuenta validada.

Personalización completa y persistente. Las paletas de color funcionan ahora en modo claro y oscuro, manteniendo el matiz característico de cada una (Sunrise sigue siendo cálida en claro, Forest sigue siendo verde). El fondo animado (Smoke, Aurora, FloatingLines, etc.) se aplica en todas las pestañas, no solo en Apariencia. El alto contraste fuerza texto negro sobre blanco (o blanco sobre negro en modo oscuro) en lugar del scheme original.

Pantalla de recuperación ante problemas de arranque. Si activas varias opciones de accesibilidad a la vez y la app no abre en los siguientes dos arranques, verás una pantalla blanca con un triángulo amarillo y la opción "Restaurar configuración por defecto". Toca ese botón y la app volverá a abrir con los valores iniciales. No es necesario borrar datos del sistema.

Filtros del mapa por compañía y línea. En el sheet de filtros del mapa, ahora hay un árbol expandible: Jerez → COMUJESA → líneas L1, L2, ... Puedes desmarcar una compañía entera o solo líneas individuales. El estado se guarda y se conserva entre sesiones.

Widgets de pantalla de inicio configurables. En Perfil → Widgets puedes configurar cada uno de los tres widgets:

- **Próximo bus:** elige línea y parada → preview en vivo del widget → "Probar" actualiza el widget de tu launcher en segundos.
- **Mi línea:** elige una línea favorita → preview con próximas salidas.
- **Saldo bonobús:** enlaza directamente al escaneo NFC; no requiere configuración.

Wizard de crear ruta con mapa interactivo. Al crear una nueva ruta como usuario normal (Perfil → Comunidad → Crear nueva ruta), el paso "Paradas" ahora abre un mapa donde tocas el punto exacto en lugar de teclear coordenadas. También puedes buscar lugares por nombre ("Hotel Jerez", "Gasolinera Cepsa") y la app los localiza automáticamente.

Click sobre línea en el mapa. Si tocas una de las líneas de bus dibujadas sobre el mapa, aparece un snackbar con el código y nombre de la línea (p. ej. "Línea L8 · Norte"). El sheet inferior se filtra para mostrar sólo esa línea y sus paradas; toca la X para volver a ver todas.

Mejor saludo en el inicio. El saludo de bienvenida ("Buenos días", "Buenas tardes", "Buenas noches", "Buena madrugada") cambia ahora cuatro veces al día y se ajusta al idioma seleccionado.

Adenda 2 — Novedades de la versión v1.12.1 (8 de junio de 2026)

Avisos aunque tengas la app cerrada. Transitly ya recibe notificaciones push reales: cuando se resuelve una incidencia que reportaste, te comparten una ruta o hay novedades de tus líneas, te llega el aviso aunque no tengas la app abierta. Puedes seguir ajustando qué categorías quieres recibir desde **Perfil** → **Notificaciones** (y las horas silenciosas).

El modo conductor sigue compartiendo con la pantalla bloqueada. Si eres conductor o conductora, al **Iniciar ruta** la app mantiene una notificación permanente y sigue compartiendo tu posición aunque bloques el móvil o cambies de aplicación. Así el resto de viajeros te ven en el mapa durante todo el trayecto. Al pulsar **Finalizar ruta**, deja de compartir y la notificación desaparece.

Planificador de viaje origen → destino. En la pestaña **Buscar** puedes indicar de dónde sales y a dónde quieres ir, y la app te propone cómo llegar, incluyendo trayectos con un transbordo.

Modo claro de la web mejorado. La página del proyecto se ve correctamente tanto en tema claro como oscuro.

Requisito actualizado: la app requiere **Android 7.0 o superior**.

08 — Presentación Final

Proyecto: Transitly (nexto-stop-v2) **Formato:** Guion de diapositivas para defensa oral del TFG **Ciclo:** Desarrollo de Aplicaciones Multiplataforma (DAM) **Defensa prevista:** semana 11 del cronograma (10-16 de junio de 2026) **Duración total estimada:** entre 18 y 22 minutos **Anclaje original:** master @ b908f3c · 2026-05-23 **Anclaje actualizado:** master @ b47180d0 · 2026-06-08 **Release pública:** v1.12.1 (APK universal) — <https://github.com/astralk9999/Transitly/releases/tag/v1.12.1>

Este documento es la base textual sobre la que se montan las diapositivas reales en la herramienta de presentación. Cada sección representa una diapositiva. El bloque **Notas** corresponde al discurso del orador.

Diapositiva 1 — Portada

- Título: **Transitly**.
- Subtítulo: aplicación multiplataforma de transporte público para ciudades medias.
- Autor del TFG y ciclo formativo: **DAM**.
- Centro y tutor o tutora.
- Fecha de defensa: junio de 2026.

Notas: saludo breve, presentación personal, anuncio de la estructura: contexto, demostración, decisiones técnicas, resultados y preguntas. (30 segundos)

Diapositiva 2 — Problema detectado

- En ciudades medias españolas no existe información oficial de transporte público en tiempo real.
- Las personas usuarias dependen de horarios impresos o de aplicaciones fragmentadas por operador.
- La experiencia se agrava con discapacidad visual, motora o cognitiva.
- Ejemplo concreto: Jerez de la Frontera, operador COMUJESA, sin app oficial con seguimiento en vivo.

Notas: plantear el problema con una pregunta sencilla al tribunal, "cuando esperais un autobus, sabeis cuanto tardara". Mencionar que el TFG nace de esa observación. (60 segundos)

Diapositiva 3 — Sector y oportunidad

- Movilidad urbana en España: aproximadamente **1.500 millones de viajes anuales en autobús urbano** (INE).
- Aproximadamente **600 millones** adicionales corresponden a transporte interurbano.
- Mercado muy fragmentado por operador, sin estándar único de datos en tiempo real.
- Oportunidad para una capa de aplicación comunitaria que cubra ese hueco hasta que GTFS-Realtime se generalice.

Notas: contextualizar el tamaño del problema, sin inflar cifras y citando la fuente. (60 segundos)

Diapositiva 4 — Demo grabada de la aplicación

- Reproducir un video de 30 segundos con la aplicación real.
- Mostrar: pantalla de inicio, mapa con autobuses, reporte de incidencia y lectura NFC.

Notas: dejar que el video hable. Comentar solo el cierre, "esto es lo que vais a ver en la demo final". (30 segundos)

Diapositiva 5 — Objetivos

- **Funcional 1.** Consultar líneas, paradas y horarios sin fricciones.
- **Funcional 2.** Reportar y consumir incidencias en comunidad.
- **Funcional 3.** Soportar tarjeta NFC del Consorcio Andalucía.
- **No funcional 1.** Accesibilidad WCAG 2.2 AA estructural.
- **No funcional 2.** Cumplimiento GDPR completo.
- **No funcional 3.** Funcionamiento sin conexión con datos precargados.

Notas: subrayar que los objetivos se definieron al inicio y se han verificado al final. (60 segundos)

Diapositiva 6 — Stack tecnológico

- **Cliente.** Flutter con Dart 3, Riverpod para estado, Hive para persistencia local.
- **Backend.** Supabase (PostgreSQL gestionado, autenticación, RLS, almacenamiento, Edge Functions).
- **Observabilidad.** Sentry para errores y PostHog para producto.
- **Mensajería.** Firebase Cloud Messaging para notificaciones push.
- **Plataformas objetivo.** Android, iOS y Web PWA.

Notas: justificar cada elección en una frase. Flutter por capa única multiplataforma, Supabase por reducir backend a configurar, Riverpod por testabilidad. (60 segundos)

Diapositiva 7 — Arquitectura

- Diagrama Mermaid con cuatro capas: presentación, dominio, infraestructura y datos.
- Organización por **feature-first**: cada funcionalidad es una carpeta autocontenida en `lib/features/`.
- 27 features funcionales actualmente.
- Aislamiento de dependencias mediante repositorios y casos de uso.

Notas: explicar la decisión de feature-first frente a layer-first, ventajas para mantenimiento y onboarding de nuevos desarrolladores. (90 segundos)

Diapositiva 8 — Funcionalidades clave

- **Mapa en vivo** con paradas y autobuses en circulación.
- **Lectura NFC** del saldo de la tarjeta del Consorcio Andalucía.
- **Reporte de incidencias** con tipología y moderación comunitaria.
- **Modo sin conexión** con región descargable.
- **Modo conductor** con código de invitación y seguimiento GPS.

Notas: acompañar cada bullet con una captura. Insistir en que las 27 features funcionan y están cubiertas por pruebas. (120 segundos)

Diapositiva 9 — Accesibilidad

- **WCAG 2.2 AA estructural** alcanzada; acta de verificación con TalkBack y VoiceOver prevista para la semana 10.
- **Daltonismo:** ocho matrices de transformación de color configurables.
- **Internacionalización:** 642 claves ARB en español, con cobertura en inglés y árabe y soporte completo de **RTL** en árabe.
- **Escala de texto y fuente para dislexia** disponibles.
- Auditoría interna con baterías automatizadas de pruebas de contraste y tamaños mínimos de objetivo táctil.

Notas: la accesibilidad no es un añadido, es una decisión desde la semana 1. Mencionar que el acta de TalkBack y VoiceOver se firma con el tutor antes de la defensa. (60 segundos)

Diapositiva 10 — Seguridad y privacidad

- **RLS en PostgreSQL con DENY-by-default** sobre todas las tablas.
- **GDPR** cubierto en sus Artículos 8, 13, 17, 20 y 21.
- **Cifrado en reposo** del almacenamiento local con HiveAesCipher.
- **Credenciales** gestionadas mediante flutter_secure_storage.
- **Banner de consentimiento** granular y opt-out por defecto en analítica, informes de fallos y comunicaciones.

Notas: explicar que el modelo de seguridad se ha diseñado para que ninguna operación sea posible sin política explícita. (90 segundos)

Diapositiva 11 — Calidad

- **679 tests** automatizados pasando (0 fallos).
- **51 migraciones SQL** consecutivas verificadas.
- **27 features** funcionales y empaquetadas (446 ficheros `.dart`, ~94k LOC).
- **8 Edge Functions** desplegadas: `send_notification`, `import_gtfs`, `delete_user`, `purge_old_data`, `generate_data_export`, `approve_user_route`, `promote_stop_to_official`, `validate_share_code`.
- **6 jobs de CI** en verde (analyze, test, build web, build APK, Semgrep, Gitleaks).
- **Scorecard:** 8,9 sobre 10 en el eje TFG y 6,0 sobre 10 en el eje Produccion.

Notas: la calidad se mide, no se afirma. La scorecard distingue deliberadamente el rigor académico del nivel de producción real, donde quedan 19 bloqueadores externos documentados. (60 segundos)

Diapositiva 12 — Metodología y gestión

- **Scrum solo adaptado:** sprints de una semana, ceremonias reducidas.
- **11 semanas** de cronograma documentado.
- **Auditorías independientes deep-dive** al cierre de cada sprint relevante.
- **Documentación como contrato verificable:** todo cambio se refleja en `docs/` y se ancla a un commit.
- **CI: 6 jobs verdes** (analyze, test, integration, build).

Notas: explicar que trabajar en solitario requiere una disciplina mayor de revisión externa. (60 segundos)

Diapositiva 13 — Bloqueadores superados

- **APK release no compilaba.** Ajuste de flags de R8 y dependencias nativas.
- **Drift documental.** Documentos divergiendo del código; se resolvió introduciendo anclajes a commit en cada documento maestro.
- **Migración SQL corrupta.** Se reconstruyó la cadena y se añadió una prueba de aplicación secuencial.
- **Accesibilidad AA verificable.** Se diseñó una batería automatizada de matrices de contraste para no depender solo del juicio humano.

Notas: referenciar los SHA reales del git log al exponer cada caso. Mostrar que son fallos resueltos, no ocultos. (90 segundos)

Diapositiva 14 — Resultados frente a objetivos

- Tabla con tres columnas: objetivo, estado y evidencia.
- Marcar **Cumplido** o **Parcial** según corresponda.
- Líneas en **Parcial** deben llevar un enlace a su bloqueador externo.
- Líneas en **Cumplido** se respaldan con el test, la migración o la Edge Function correspondientes.

Notas: transparencia. Lo parcial es parcial. (60 segundos)

Diapositiva 15 — Lecciones aprendidas

- **La documentación solo vale si es contrato verificable.** Cada decisión tiene que apuntar a un commit, una prueba o un documento.
- **Una auditoría independiente vale más que cien autorrevisiones.**
- **Las inteligencias artificiales colaboran bien, pero exigen governance.** Decisiones, prompts y revisiones están documentadas.
- **El alcance se gestiona acotando, no estirando.** Mejor 27 features sólidas que 50 a medias.

Notas: estas son las cuatro lecciones que se llevaría a futuros proyectos. (60 segundos)

Diapositiva 16 — Trabajo futuro

- Integración con **GTFS-Realtime** cuando los operadores expongan APIs públicas.
- Soporte completo de **WidgetKit en iOS** equivalente a los widgets Android ya existentes.
- Modelo de **aprendizaje automático para predicción de ETA**, entrenado con históricos de GPS de conductor.
- **Expansion multi-ciudad** más allá del piloto en Jerez.

Notas: dejar claro que el trabajo futuro existe porque hay una base sólida sobre la que construirlo. (60 segundos)

Diapositiva 17 — Conclusiones

- Transitly demuestra que es viable construir, en solitario y con asistencia de IA documentada, una aplicación móvil profesional con exigencias reales de accesibilidad y privacidad.
- La arquitectura feature-first y la disciplina documental han sido las decisiones de mayor retorno.
- El TFG se entrega con metrica reproducible: 679 tests, 51 migraciones, 27 features, 8 Edge Functions y una release pública instalable (v1.12.1).

Notas: cerrar con una frase memorable, "Transitly no es una maqueta, es un sistema verificable". (45 segundos)

Diapositiva 18 — Demostracion en vivo

- Reproducir el flujo guiado de la aplicación en el dispositivo real.
- Recorrido: alta como invitado, consulta de línea, mapa, reporte de incidencia, lectura NFC simulada, descarga offline y cambio de idioma a arabe (RTL).
- Plan B: en caso de fallo de red, ejecutar todo el flujo en modo offline.

Notas: cinco minutos planificados, con un guion impreso de apoyo y un plan B documentado. (300 segundos)

Diapositiva 19 — Preguntas y respuestas

- Tribunal: cinco minutos para preguntas abiertas.
- Recursos preparados: scorecard, mega plan, registro de bloqueadores externos y política de privacidad.
- Disposición para profundizar en cualquier feature concreta o en la arquitectura.

Notas: mantener calma, repetir la pregunta antes de contestarla, no rellenar con suposiciones. (300 segundos)

Diapositiva 20 — Agradecimientos

- Al tutor o tutora del TFG por la orientacion y revisiones.
- A la familia por el sostenimiento durante los meses de trabajo.
- A las companeras y companeros del ciclo por las revisiones cruzadas.

- A la comunidad de software libre y a los proyectos Flutter, Supabase, Sentry y PostHog cuyo trabajo hace posible Transitly.

Notas: cerrar con una frase breve de cierre y dar paso al tribunal. (30 segundos)

Diapositivas adicionales — Versión 1.11.0 (4 de junio de 2026)

Estas diapositivas se insertan entre la **slide 8 (Demostración)** y la **slide 9 (Resultados y métricas)** del guion original. Refuerzan el bloque de "trabajo realizado entre el cierre del anchor original y la defensa".

Slide 8.bis — Estabilización post-MVP en cifras

Título: Del MVP al release público — 12 días, 94 commits, una versión instalable.

Cuerpo:

- 94 commits entre 23/05 y 04/06 organizados en cinco oleadas de estabilización.
- Cuatro planes de acción ejecutados (`PLAN_15_BUGS` , `PLAN_8_BUGS_LOGS` , `PLAN_CRASH_NATIVO_RECOVERY` , `PLAN_21_BUGS_MAPA_PERFIL` , `PLAN_8_MEJORAS` , `PLAN_42P17_RLS_RECURSION`).
- Release oficial v1.11.0 publicado en GitHub Releases con APK firmado.
- Una migración SQL adicional (`fix_route_shares_rls_recursion`).
- 18 nuevas claves ARB (widgets config, avisos de zona).
- Cinco features nuevas: widgets configurables, wizard con mapa, árbol de filtros, recovery boot, tile prewarming.

Notas para la defensa: explicar que el ciclo de estabilización post-MVP no estaba contemplado con detalle en el plan original pero se absorbió dentro del margen de la semana 10. Subrayar que cada plan se documenta como `PLAN_*.md` en `docs/historico/` , con causa raíz, decisiones tomadas y criterios de aceptación verificables. (90 segundos)

Slide 8.ter — Bug crítico que la app sobrevivió por sí sola

Título: Boot canary — cómo una app puede recuperarse sola sin clear data.

Cuerpo:

Caso real detectado en pruebas con dispositivos físicos:

1. Usuario activa simultáneamente: dislexia + alto contraste + tamaño de texto máximo + filtro de daltonismo.
2. La app crashea en el siguiente arranque por una combinación tóxica en el subsistema nativo de Flutter (font + shader + matrix de color).
3. Las defensas Dart (`try/catch` , validación de NaN, fallbacks) **no capturan** el crash: ocurre en C++, no en Dart.
4. El estado tóxico ya está persistido en Hive → al reabrir la app vuelve a crashear → bucle.

Solución implementada:

- `BootCanary` marca `BOOTING` al inicio de `main()` y lo cambia a `STABLE` tras el primer `addPostFrameCallback`.
- Si el siguiente arranque ve `BOOTING` → la app crasheó antes del primer frame → se incrementa el contador.
- Se revierte la última preferencia sensible (sólo persistimos preferencias después de `markStable`).
- Tras dos crashes consecutivos, se monta `RecoveryScreen` con `MaterialApp` propio, **sin shaders ni fuentes custom**, con botón "Restaurar valores por defecto".

Resultado: indicador "crashes que requieren clear data" pasa de cualquier valor a **cero**. La aplicación se vuelve auto-recuperable.

Notas para la defensa: este es uno de los puntos más diferenciadores del proyecto. Resiliencia ante errores nativos no es algo que se enseñe en el ciclo y demuestra capacidad de razonamiento sobre la diferencia entre Dart y el engine subyacente. (120 segundos)

Slide 8.querter — Detección y resolución de un ciclo recursivo en RLS

Título: Error 42P17 — "infinite recursion in policy". Un caso real de auditoría SQL.

Cuerpo:

Tras conseguir que el login con Google funcionara, el flujo post-login fallaba al cargar las contribuciones del usuario:

```
PostgresException: infinite recursion detected in policy for relation "route_shares"
code: 42P17
```

Auditoría con MCP de Supabase (consulta a `pg_policies`):

- `route_shares.route_shares_select_owner` hacía `EXISTS (SELECT FROM routes WHERE owner_id = auth.uid())`.
- `routes.routes_select_visible` hacía `EXISTS (SELECT FROM route_shares WHERE shared_with_id = auth.uid())`.
- PostgreSQL detecta el ciclo → aborta.

Solución desplegada (migración `fix_route_shares_qls_recursion`):

```
CREATE FUNCTION public.is_route_owner(p_route_id uuid)
RETURNS boolean LANGUAGE sql SECURITY DEFINER STABLE
AS $$ SELECT EXISTS (SELECT 1 FROM routes WHERE id = p_route_id AND owner_id = auth.uid());
$$;
```

La función ejecuta con permisos de owner (bypassa RLS) y rompe el ciclo. Las tres policies que consultaban `routes` (SELECT, DELETE, INSERT del owner) llaman ahora a la función. Semántica de visibilidad preservada.

Notas para la defensa: explicar conceptualmente qué es Row Level Security y por qué un sistema multi-rol como Transitty lo necesita. Mencionar que la decisión de mantener el patrón `SECURITY`

`DEFINER` versus desnormalizar el schema es un trade-off documentado en `docs/SUPABASE_SETUP.md`. (90 segundos)

Slide 8.quinta — Release pública y limpieza del repositorio

Título: De `presentation/public/*.apk` a GitHub Releases — 792 MB liberados.

Cuerpo:

Estado anterior al 04/06:

- 9 APKs históricos versionados en `presentation/public/transitly-v1.{3..11}.0.apk`.
- ~792 MB en el `working tree`.
- GitHub avisaba `>50 MB` por archivo y bloquearía con `>100 MB`.
- Cada nuevo APK duplicaba ~88 MB en el `.git/` indefinidamente.

Acción tomada:

1. Eliminar los 9 APKs del HEAD del repositorio.
2. Añadir patrón `*.apk` al `.gitignore` raíz.
3. Cambiar las URLs de descarga en la presentación web a `https://github.com/astralk9999/Transitly/releases/latest` (URL estable que GitHub redirige al último release).
4. Crear el release oficial `v1.11.0` con el APK como asset usando `gh release create`.

Resultado:

- `working tree` reducido en ~792 MB.
- Distribución profesional vía Releases (límite 2 GB por archivo, sin contaminar el repo).
- Una sola URL que la presentación nunca tiene que actualizar.

Notas para la defensa: mencionar que esto se descubrió en el aviso de GitHub al hacer `git push` y se decidió aplicar la práctica estándar del ecosistema (Releases para binarios, repo sólo para código). La presentación HTML usaba `import.meta.env.BASE_URL` que apuntaba a `/Transitly/<archivo>.apk`; la sustitución por `releases/latest` deja la web independiente del versionado. (60 segundos)

Diapositivas adicionales — Versión 1.12.1 (8 de junio de 2026)

Estas diapositivas cierran el bloque de "trabajo realizado hasta la defensa" y se insertan tras la slide 8.quinta. Demuestran que los objetivos funcionales 2, 7 y 12 quedan totalmente cubiertos en la release final.

Slide 8.sexta — Notificaciones push reales con la app cerrada

Título: De la notificación local al push real — Firebase Cloud Messaging extremo a extremo.

Cuerpo:

- **Ciente:** integración completa de FCM (proyecto `transitly-ee8cf`): `google-services.json`, plugin Gradle, `firebase_options.dart` con claves reales, manejadores de mensajes en primer y segundo plano y registro del token del dispositivo en `device_tokens` al iniciar sesión.
- **Servidor (ya existente):** Edge Function `send_notification` que firma un JWT OAuth y envía por **FCM HTTP v1**, con limpieza automática de tokens inválidos, disparada por triggers SQL (incidencia resuelta, ruta compartida, ruta promovida).
- **Resultado verificado:** la app recibe push **con la app cerrada** (`FlutterFirebaseMessagingBackgroundService started`).
- **Dependencia externa documentada:** la *service account* del operador para el envío programático (`docs/FCM_SETUP.md`).

Notas para la defensa: distinguir entre la config de **cliente** (`google-services.json`) y la credencial de **servidor** (service account), y por qué el envío programático depende de esta última. (90 segundos)

Slide 8.séptima — Modo conductor en segundo plano (foreground service)

Título: Compartir posición con la pantalla bloqueada — un foreground service de Android.

Cuerpo:

- El seguimiento del conductor pasó de un `Timer` en primer plano a `Geolocator.getPositionStream` con **foreground service** (`ForegroundNotificationConfig`, wake-lock, notificación persistente nativa).
- La posición se sigue emitiendo al canal Realtime con la app en segundo plano o el móvil bloqueado; al terminar la ruta se cancela el stream y la notificación.
- Permisos `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION` y `WAKE_LOCK` en el manifiesto. En web (sin foreground service) se mantiene el envío periódico mientras la pestaña está activa.

Notas para la defensa: explicar por qué un `Timer` se pausa al bloquear la pantalla y cómo el foreground service garantiza la continuidad, requisito real para un conductor en ruta. (75 segundos)

Slide 8.octava — Planificador origen→destino reactivado

Título: Cerrar el bucle de consulta — trayectos con transbordos.

Cuerpo:

- Reactivación de la pestaña **Buscar** y del flujo origen→destino (`RoutePlannerService`: rutas directas y con un transbordo) navegando a `/route-plan`.
- Completa la carencia informativa: el viajero no solo ve líneas y paradas, sino cómo ir de A a B.

Notas para la defensa: mencionar que el motor de planificación es heurístico sobre el grafo de paradas y que la integración con horarios reales es la línea de evolución natural. (45 segundos)

Slide 8.novena — La web del proyecto en GitHub Pages

Título: Distribución y documentación pública.

Cuerpo:

- Sitio público en **GitHub Pages** (<https://astralk9999.github.io/Transitly/>) con la presentación del proyecto, los **entregables del TFG** navegables y la **descarga del APK** resuelta automáticamente desde la release más reciente.
- Release **v1.12.1**: APK **universal** (arm64-v8a, armeabi-v7a, x86_64), Android 7.0+.
- La web de producto (landing + app Flutter embebida en [/app](#)) se ejecuta en local para la demo, con la app Android como eje central de la defensa.

Notas para la defensa: señalar que la página sirve a la vez de portfolio, de canal de distribución y de índice documental del TFG. (45 segundos)